

COMPENDIO TEÓRICO COMPLETO

TÉCNICAS DIGITALES II — 1ER PARCIAL TEÓRICO

Universidad Tecnológica Nacional — Facultad Regional Córdoba — Cátedra: Dr. Ing. Steiner Guillermo

Hardware: STM32F103C8T6 (ARM Cortex-M3 @ 72 MHz, 64 KB Flash, 20 KB SRAM) · Toolchain: GNU GCC · Kernel: FreeRTOS v10.x

Índice

1. Módulo 1: Arquitectura de Computadores, Rendimiento y Hardware/Software	2
1.1. Sistemas Embebidos vs Computación General	2
1.2. Unidades de Almacenamiento: Estándar Decimal (SI) vs Binario (IEC 80000-13)	2
1.3. Las 8 Grandes Ideas de la Arquitectura de Computadores	2
1.4. La Interfaz Hardware/Software: ISA y ABI	2
1.5. Ecuación Fundamental del Rendimiento de CPU	2
1.6. El Muro de Potencia (Power Wall) y la Ley de Amdahl	3
2. Módulo 2: Arquitectura ARMv7-M, Núcleo Cortex-M3 y Mapeo de Memoria	3
2.1. Comparación Detallada: Arquitecturas RISC vs CISC	3
2.2. El Núcleo ARM Cortex-M3 y el Juego Thumb-2	3
2.3. Modelo de Registros del Programador (Core Registers)	3
2.4. Modos de Operación y Niveles de Privilegio	4
2.5. Mapeo de Memoria y Decodificación de Direcciones	4
2.6. El Calificador 'volatile' en C para Acceso a Registros	4
2.7. Mecanismo de Bit-Banding (Operaciones Atómicas de 1 Bit)	5
3. Módulo 3: Cadena de Construcción (Toolchain), Enlazado y Arranque	5
3.1. Las Cuatro Fases del Toolchain de Compilación	5
3.2. Estructura de Secciones de un Archivo Objeto y Ejecutable (ELF)	5
3.3. Dualidad LMA vs VMA para la Sección .data	6
3.4. El Linker Script (.ld) en Detalle	6
3.5. El Código de Arranque (Startup / Reset_Handler)	6
4. Módulo 4: Controlador de Interrupciones y Excepciones (NVIC)	6
4.1. Concepto y Clasificación de Excepciones	7
4.2. La Tabla de Vectores y Reubicación Dinámica (VTOR)	7
4.3. Esquema de Prioridades del NVIC: Grupo vs Sub-prioridad	7
4.4. Mecanismo de Auto-Stacking por Hardware y EXC_RETURN	7
4.5. Optimizaciones de Latencia del NVIC	7
4.6. Registros de Enmascaramiento Global del Núcleo	8
4.7. El Controlador de Interrupciones Externas (EXTI) y Mapeo AFIO	8
5. Módulo 5: Capas de Activación y Protocolos de Comunicación Serie	8
5.1. Las 4 Capas Jerárquicas de Activación de Periféricos en STM32	8
5.2. Sistema de Relojes (RCC) y el Clock Tree del STM32F103	8
5.3. Capa GPIO: Decodificación de CNF y MODE	8
5.4. Protocolo SPI (Serial Peripheral Interface)	9
5.5. Protocolo I2C (Inter-Integrated Circuit)	9
5.6. Protocolo UART / USART y Estándar RS-232	10
5.7. Gran Cuadro Comparativo de Protocolos Serie: SPI vs I2C vs UART	11
6. Módulo 6: Sistemas Operativos en Tiempo Real (FreeRTOS)	11
6.1. Paradigma Super-Loop (Bare-Metal) vs RTOS	11
6.2. Conceptos de Tiempo Real: Hard vs Soft Real-Time	12
6.3. Arquitectura, Tipos de Datos y Notación Húngara en FreeRTOS	12
6.4. Gestión de Tareas (Task Management)	12
6.5. El Planificador (Scheduler) y Algoritmo de Selección	13
6.6. Temporización: vTaskDelay() vs vTaskDelayUntil()	13
6.7. La Tarea Idle y la Función Idle Hook	13
6.8. Mecanismos de Comunicación: Colas de Mensajes (Queues)	13
6.9. Gestión de Interrupciones en FreeRTOS y Procesamiento Diferido	14
7. Módulo 7: Banco de 30 Preguntas Típicas de Examen Teórico con Respuestas Modelo	14
8. Módulo 8: Actividades y Ejercicios Extraídos de la Bibliografía Oficial	20
8.1. Fuente 1: Patterson & Hennessy — <i>Computer Organization and Design (ARM Edition)</i>	20
8.2. Fuente 2: Harris & Harris — <i>Digital Design and Computer Architecture (ARM Edition)</i>	20
8.3. Fuente 3: Joseph Yiu — <i>The Definitive Guide to ARM Cortex-M3 and Cortex-M4</i>	21
8.4. Fuente 4: Richard Barry — <i>Mastering the FreeRTOS Real Time Kernel</i>	21
9. Módulo 9: Solucionario Oficial de las Actividades de la Bibliografía	22
9.1. Soluciones: Patterson & Hennessy (<i>Computer Organization and Design</i>)	22
9.2. Soluciones: Harris & Harris (<i>Digital Design and Computer Architecture</i>)	23
9.3. Soluciones: Joseph Yiu (<i>The Definitive Guide to ARM Cortex-M3/M4</i>)	24
9.4. Soluciones: Richard Barry (<i>Mastering the FreeRTOS Real Time Kernel</i>)	24

1 Módulo 1: Arquitectura de Computadores, Rendimiento y Hardware/Software

1.1 Sistemas Embebidos vs Computación General

Un **Sistema Embebido** es un sistema computacional diseñado para realizar una o pocas funciones dedicadas, frecuentemente dentro de un sistema mecánico o eléctrico mayor. El software forma una unidad insoluble con el hardware.

Restricciones Fundamentales de Diseño en Sistemas Embebidos

1. **Costo Extremo:** Producidos en tiradas de cientos de miles o millones de unidades. Cada centavo de dólar en el microcontrolador, memorias o pasivos impacta directamente en la viabilidad económica del producto.
2. **Consumo de Potencia y Energía:** Crítico en dispositivos alimentados a batería (sensores remotos, wearables) y en entornos industriales cerrados con refrigeración puramente pasiva (sin ventiladores).
3. **Confiabilidad, Disponibilidad y Seguridad Funcional (Safety-Critical):** En sistemas críticos (frenos ABS, control de aviónica, marcapasos, control de inyección automotriz), un fallo del sistema puede ocasionar catástrofes humanas o pérdidas millonarias.
4. **Determinismo Temporal y Tiempo Real (Real-Time):** Capacidad de responder a estímulos físicos del entorno garantizando un tiempo de respuesta acotado y predecible (deadline).

1.2 Unidades de Almacenamiento: Estándar Decimal (SI) vs Binario (IEC 80000-13)

Históricamente existió ambigüedad entre las unidades de base 10 (fabricantes de discos) y de base 2 (diseñadores de memoria y software). La norma IEC 80000-13 formalizó los prefijos binarios:

Prefijo SI	Símb.	Valor Base 10	Prefijo IEC	Símb.	Valor Base 2	Diferencia
Kilobyte	KB	$10^3 = 1,000 \text{ B}$	Kibibyte	KiB	$2^{10} = 1,024 \text{ B}$	+2,4 %
Megabyte	MB	$10^6 = 1,000,000 \text{ B}$	Mebibyte	MiB	$2^{20} = 1,048,576 \text{ B}$	+4,86 %
Gigabyte	GB	$10^9 = 10^9 \text{ B}$	Gibibyte	GiB	$2^{30} = 1,073,741,824 \text{ B}$	+7,37 %
Terabyte	TB	$10^{12} = 10^{12} \text{ B}$	Tebibyte	TiB	$2^{40} = 1,099,511,627,776 \text{ B}$	+9,95 %

Importancia en TD2: Una memoria Flash de 64 KiB en el STM32F103 posee exactamente $64 \times 1,024 = 65,536$ bytes, direccionables mediante 16 líneas de dirección ($2^{16} = 65,536$).

1.3 Las 8 Grandes Ideas de la Arquitectura de Computadores

Formuladas por David Patterson y John Hennessy, constituyen los pilares del diseño digital moderno:

1. **Diseño para la Ley de Moore:** Como el ciclo de diseño dura de 2 a 4 años, se debe proyectar el chip para la tecnología que existirá al momento de fabricación, no la del inicio.
2. **Uso de la Abstracción para Simplificar el Diseño:** Ocultar detalles de bajo nivel mediante capas (Hardware → Microarquitectura → ISA → OS/Drivers → Aplicación).
3. **Hacer el Caso Común Rápido (Make the Common Case Fast):** Optimizar las operaciones que ocurren con mayor frecuencia (ej. instrucciones simples en RISC).
4. **Rendimiento Mediante Paralelismo:** Ejecutar múltiples operaciones simultáneamente (multiprocesadores, multicore).
5. **Rendimiento Mediante Segmentación (Pipelining):** Solapar etapas de ejecución de distintas instrucciones (Fetch, Decode, Execute).
6. **Rendimiento Mediante Predicción:** Anticipar el flujo de ejecución (ej. Branch Prediction).
7. **Jerarquía de Memorias:** Combinar memorias pequeñas y ultrarrápidas (Registros, Caché) con memorias más grandes y lentas (SRAM, Flash, Disco).
8. **Confiabilidad Mediante Redundancia:** Incluir componentes de respaldo (ECC en memoria, paridad, watchdog) para tolerar fallos.

1.4 La Interfaz Hardware/Software: ISA y ABI

- **ISA (Instruction Set Architecture):** Contrato abstracto entre el hardware y el software. Define el conjunto de instrucciones de máquina, registros visibles, modos de direccionamiento, modelo de memoria y excepciones. Permite que múltiples implementaciones físicas (ej. Cortex-M3, Cortex-M4, Apple Silicon) ejecuten el mismo código binario.
- **Microarquitectura:** Implementación física concreta de la ISA en silicio (tamaño de pipeline, buses internos, unidades funcionales ALU/Multiplicador).
- **ABI (Application Binary Interface):** Reglas a nivel binario para que módulos compilados por separado interactúen (convenciones de llamada, pasaje de parámetros en registros R0-R3, alineación de stack). En ARM se utiliza el estándar **AAPCS** (ARM Architecture Procedure Call Standard).

1.5 Ecuación Fundamental del Rendimiento de CPU

El rendimiento de un sistema se evalúa mediante el tiempo de ejecución de un programa:

Ecuación de Tiempo de CPU

$$\text{Tiempo de CPU} = \text{Instrucciones del Programa (IC)} \times \text{CPI} \times T_{\text{clock}} = \frac{\text{IC} \times \text{CPI}}{f_{\text{clock}}}$$

donde:

- **IC (Instruction Count):** Cantidad total de instrucciones ejecutadas por el programa. Depende del algoritmo, compilador e ISA.
- **CPI (Cycles Per Instruction):** Promedio de ciclos de reloj necesarios para ejecutar una instrucción. Depende de la microarquitectura y la ISA.
- $T_{\text{clock}} = \frac{1}{f_{\text{clock}}}$: Período de la señal de reloj del procesador. Depende de la tecnología de fabricación y la lógica del chip.

$$\text{CPI Ponderado} = \sum_{i=1}^n (\text{CPI}_i \times f_i) = \frac{\sum (\text{CPI}_i \times \text{Count}_i)}{\text{IC}}$$

1.6 El Muro de Potencia (Power Wall) y la Ley de Amdahl

Durante décadas, la frecuencia de reloj creció exponencialmente gracias al escalamiento de Dennard. Hacia 2006, la industria se topó con el **Muro de Potencia**, imposibilitando seguir aumentando los GHz en un solo núcleo.

Potencia Dinámica en CMOS y Ley de Amdahl

$$P_{\text{din}} = \frac{1}{2} \cdot C_{\text{carga}} \cdot V_{DD}^2 \cdot f_{\text{clock}} \cdot \alpha$$

donde C es la capacitancia parásita, V_{DD} la tensión de alimentación, f la frecuencia y α el factor de actividad.

Límites Físicos:

1. **Límite de Voltaje:** Reducir V_{DD} por debajo de $\sim 0,8\text{V}$ dispara las corrientes de fuga (*leakage currents*) y el ruido cuántico.
2. **Límite Térmico:** La densidad de potencia superó los 100 W/cm^2 (densidad térmica de la tobera de un cohete), requiriendo soluciones de disipación inviables.

Solución de la Industria: Giro hacia procesadores **Multi-núcleo (Multicore)** a frecuencias moderadas.

Ley de Amdahl (Límite del Aceleramiento Paralelo):

$$S_{\text{total}} = \frac{1}{(1 - F) + \frac{F}{S}}$$

donde F es la fracción del código paralelizable y S el factor de aceleración de esa fracción. Si el 20 % del código es puramente secuencial ($F = 0,8$), el aceleramiento máximo teórico con infinitos núcleos es $\frac{1}{1-0,8} = 5\times$.

2 Módulo 2: Arquitectura ARMv7-M, Núcleo Cortex-M3 y Mapeo de Memoria

2.1 Comparación Detallada: Arquitecturas RISC vs CISC

Característica	RISC (ARM, MIPS, RISC-V)	CISC (x86, 8051, PIC clásico)
Conjunto de Instrucciones	Reducido, simple y ortogonal. Instrucciones de longitud fija (16 o 32 bits).	Complejo y extenso (cientos de variantes). Instrucciones de longitud variable (1 a 15 bytes).
Acceso a Memoria	Arquitectura Load/Store: Únicamente las instrucciones LDR y STR tocan la RAM. Las operaciones aritméticas operan exclusivamente sobre registros.	Operaciones aritméticas pueden operar directamente sobre operandos en memoria (ej. ADD [EAX], EBX).
Ciclos por Instrucción (CPI)	Muy cercano a 1 (mayoría de instrucciones ejecutan en 1 ciclo de reloj).	Variable y alto (instrucciones complejas toman entre 2 y decenas de ciclos).
Pipeline (Segmentación)	Fácil de implementar y altamente eficiente debido a la longitud y tiempos uniformes.	Complejo de segmentar por la longitud variable y microoperaciones internas.
Filosofía de Optimización	Traslada la complejidad al Compilador , simplificando el silicio para ahorrar área y consumo.	Traslada la complejidad al Hardware (decodificadores microprogramados complejos).

2.2 El Núcleo ARM Cortex-M3 y el Juego Thumb-2

El Cortex-M3 implementa la arquitectura **ARMv7-M**. Opera exclusivamente con el conjunto de instrucciones **Thumb-2**, que combina instrucciones de 16 bits (alta densidad de código) e instrucciones de 32 bits (alto rendimiento) sin necesidad de conmutar modos de CPU.

- **Arquitectura Harvard Modificada:** Posee buses separados para instrucciones (I-Code) y datos (D-Code) hacia la memoria interna, y un bus de Sistema (S-Bus) para periféricos y SRAM, permitiendo accesos simultáneos sin cuellos de botella de Von Neumann.
- **Pipeline de 3 etapas:** Fetch (Búsqueda), Decode (Decodificación) y Execute (Ejecución).

2.3 Modelo de Registros del Programador (Core Registers)

El Cortex-M3 posee un banco de registros de 32 bits accesibles por software:

Registro	Nombre	Función y Características
R0 – R7	Registros Bajos (Low Registers)	Accesibles por todas las instrucciones Thumb de 16 y 32 bits. Parámetros y retorno en C (R0-R3).
R8 – R12	Registros Altos (High Registers)	Registros de propósito general de 32 bits.
R13 / SP	Stack Pointer (Puntero de Pila)	Físicamente desdoblado en dos registros (bancados): MSP (Main) y PSP (Process).
R14 / LR	Link Register (Registro de Enlace)	Almacena la dirección de retorno de subrutinas (BL) o el código EXC_RETURN en excepciones.
R15 / PC	Program Counter	Apunta a la instrucción a ejecutar. Bit 0 debe ser 1 para indicar ejecución en modo Thumb.
xPSR	Program Status Register	Registro de estado compuesto por tres vistas: APSR, IPSR y EPSR.

Estructura del Registro de Estado xPSR

El registro xPSR es una vista combinada de tres registros de estado:

1. **APSR (Application PSR):** Bits [31:27] contienen los flags de condición aritmética:
 - **N (Negative):** 1 si el resultado de la operación es negativo (bit 31 = 1).
 - **Z (Zero):** 1 si el resultado de la operación es exactamente cero.
 - **C (Carry):** 1 si hubo acarreo en una suma o no hubo préstamo en una resta.
 - **V (Overflow):** 1 si hubo desbordamiento en complemento a dos con signo.
 - **Q (Saturation):** Indica saturación en operaciones matemáticas DSP/saturantes.
2. **IPSR (Interrupt PSR):** Bits [8:0] contienen el número de excepción activa actual (0 = Modo Thread / no hay excepción; 15 = SysTick; 16+ = IRQ de periférico).
3. **EPSR (Execution PSR):** Contiene el bit **T (Thumb, bit 24)**, que debe ser siempre 1. Si se intenta ejecutar una instrucción con $T = 0$, el procesador entra inmediatamente en fallo de hardware (HardFault o UsageFault).

2.4 Modos de Operación y Niveles de Privilegio

Cortex-M3 define una matriz de seguridad de 2×2 entre modos y privilegios:

Nivel de Privilegio	Thread Mode (Modo Hilo)	Handler Mode (Modo Manejador)
Privileged (Privilegiado)	Modo de ejecución normal del sistema operativo o aplicaciones Bare-Metal simples. Acceso irrestricto a todo el mapa de memoria y registros de control. Usa MSP o PSP.	Modo en que se ejecutan TODAS las excepciones e interrupciones (ISRs). Siempre es privilegiado. Usa exclusivamente MSP.
Unprivileged / User (No Privilegiado)	Ejecución de tareas de usuario en un RTOS. Acceso bloqueado a la región PPB (0xE000_0000, NVIC, SysTick) y restringido por MPU. Usa típicamente PSP.	No permitido. Las interrupciones nunca se ejecutan en modo no privilegiado.

El Registro de Control (CONTROL)

El registro especial CONTROL posee dos bits fundamentales configurables por software:

- **Bit 0 (nPRIV):** 0 = Nivel Privilegiado; 1 = Nivel No Privilegiado (Usuario).
- **Bit 1 (SPSEL):** 0 = Utiliza el Main Stack Pointer (MSP); 1 = Utiliza el Process Stack Pointer (PSP) en Thread Mode.

Transición de Seguridad: Un software no privilegiado NO puede escribir CONTROL para volver a ser privilegiado. La única forma de recuperar privilegios es solicitar un servicio al sistema operativo mediante la instrucción de interrupción por software SVC (Supervisor Call), la cual entra a Handler Mode en modo privilegiado.

2.5 Mapeo de Memoria y Decodificación de Direcciones

El Cortex-M3 posee un bus de direcciones de 32 bits que define un espacio plano y continuo de **4 GiB** (0x0000_0000 a 0xFFFF_FFFF):

Región	Rango de Direcciones	Tamaño	Uso y Asignación en STM32F103
Code / Flash	0x0000_0000 - 0x1FFF_FFFF	512 MB	Memoria Flash del programa (0x0800_0000), Boot ROM (0x1FFF_F000).
SRAM	0x2000_0000 - 0x3FFF_FFFF	512 MB	Memoria RAM estática interna (0x2000_0000, 20 KB en BluePill).
Peripherals	0x4000_0000 - 0x5FFF_FFFF	512 MB	Registros de periféricos en buses APB1, APB2 y AHB (GPIO, ADC, TIM, USART, etc.).
External RAM	0x6000_0000 - 0x9FFF_FFFF	1 GB	Memorias externas FSMC (NOR, NAND, SRAM externa).
External Device	0xA000_0000 - 0xDFFF_FFFF	1 GB	Dispositivos externos mapeados.
Private Periph.	0xE000_0000 - 0xE00F_FFFF	1 MB	PPB (Core): NVIC (0xE000_E100), SysTick (0xE000_E010), SCB (0xE000_ED00), MPU.

¿Por qué el NVIC y SysTick están en el PPB (0xE000_0000)?

El bus PPB (Private Peripheral Bus) es un bus interno exclusivo de ARM que conecta el núcleo directamente con el NVIC y SysTick.

1. **Latencia Mínima y Determinismo:** Las operaciones sobre el NVIC y SysTick no compiten con las transferencias de periféricos en los buses AHB/APB.
2. **Seguridad y Privilegio:** El hardware bloquea automáticamente cualquier intento de acceso al PPB desde software no privilegiado, generando un fallo de bus.
3. **Portabilidad Absoluta (CMSIS):** Al estar ubicados en las mismas direcciones en cualquier chip ARM Cortex-M3 (ST, NXP, TI, Microchip), el código de control de interrupciones es 100 % portable.

2.6 El Calificador 'volatile' en C para Acceso a Registros

En C, el calificador volatile le indica al compilador que el valor de una variable o posición de memoria puede ser modificado por eventos externos al flujo del programa (hardware, interrupciones o DMA).

!! ATENCION: ¿Qué ocurre si NO se usa 'volatile'?

El optimizador del compilador (GCC con -O2 o -O3) asume que si el código no modifica una variable en un bucle, su valor nunca cambia. Por lo tanto, almacena el valor en un registro de la CPU (*caching en R0-R12*) y nunca vuelve a leer la memoria física.

Ejemplo de fallo fatal sin volatile:

```
// Espera a que el ADC termine la conversion (flag EOC en bit 1)
#define ADC_SR (*(uint32_t *)0x40012400) // ERROR: Falta volatile
while (!(ADC_SR & (1 << 1))); // El compilador lee ADC_SR UNA SOLA VEZ y entra en bucle infinito.

// Declaracion correcta:
#define ADC_SR (*(volatile uint32_t *)0x40012400) // Correcto: lee la direccion fisica en cada vuelta
```

2.7 Mecanismo de Bit-Banding (Operaciones Atómicas de 1 Bit)

Cortex-M3 introduce la técnica de **Bit-Banding**, que mapea cada bit individual de una región de 1 MB (*Bit-Band Region*) a una palabra completa de 32 bits en otra región de 32 MB (*Bit-Band Alias*).

- **Ventaja Crítica:** Permite modificar un único bit en un periférico o variable en RAM mediante una sola instrucción de escritura (STR), garantizando **atomicidad por hardware** (libre de condiciones de carrera con interrupciones) sin necesidad de instrucciones de deshabilitación global de interrupciones.
- **Regiones Soportadas:**
 1. **SRAM Bit-Band:** Región base 0x2000_0000 (1 MB) → Alias en 0x2200_0000 (32 MB).
 2. **Peripheral Bit-Band:** Región base 0x4000_0000 (1 MB) → Alias en 0x4200_0000 (32 MB).

Fórmula de Cálculo de Dirección de Bit-Band Alias

$$\text{Alias_Address} = \text{Alias_Base} + (\text{Byte_Offset} \times 32) + (\text{Bit_Number} \times 4)$$

Ejemplo: Modificar el bit 4 del puerto GPIOA ODR (0x4001_080C):

- Alias_Base = 0x4200_0000
- Byte_Offset = 0x4001_080C – 0x4000_0000 = 0x1080C = 67,596
- Bit_Number = 4
- Alias_Address = 0x4200_0000 + (67,596 × 32) + (4 × 4) = 0x4200_0000 + 0x210180 + 0x10 = 0x4221_0190
- Escribir un 1 en *(volatile uint32_t *)0x42210190 = 1; pone en 1 el bit 4 de forma puramente atómica.

3 Módulo 3: Cadena de Construcción (Toolchain), Enlazado y Arranque**3.1 Las Cuatro Fases del Toolchain de Compilación**

La transformación de código fuente en C (.c) a un archivo binario ejecutable en el microcontrolador involucra cuatro etapas secuenciales coordinadas por la suite GNU GCC (arm-none-eabi):

Fase	Herramienta	Entrada → Salida	Responsabilidad y Operación Interna
1. Preprocesamiento	Preprocesador (cpp)	.c, .h → .i	Expande directivas #include , reemplaza macros #define y resuelve bloques condicionales (#ifdef , #ifndef).
2. Compilación	Compilador (cc1)	.i → .s	Análisis léxico, sintáctico y semántico. Genera árbol AST, optimiza código (-O2, -Os) y emite código ensamblador específico de ARMv7-M .
3. Ensamblado	Ensamblador (as)	.s → .o	Traduce mnemónicos a código máquina binario. Opera en dos pasadas generando un archivo objeto en formato ELF (<i>Executable and Linkable Format</i>).
4. Enlazado	Enlazador (ld)	.o, .a, .ld → .elf	Concatena secciones homogéneas, resuelve referencias cruzadas externas y asigna direcciones de memoria física definitivas.

¿Por qué el Ensamblador requiere Dos Pasadas?

1. **Primera Pasada (Construcción de la Tabla de Símbolos):** Lee el archivo línea por línea y mantiene un Contador de Posición (*Location Counter*). Registra todas las etiquetas (nombres de funciones, variables) y les asocia un offset relativo. No puede resolver saltos hacia adelante (*forward references*).
2. **Segunda Pasada (Emisión de Código Máquina):** Traduce cada instrucción mnemónica a su opcode binario exacto, reemplazando todas las etiquetas por los offsets calculados en la primera pasada y emitiendo el archivo objeto relocable .o.

3.2 Estructura de Secciones de un Archivo Objeto y Ejecutable (ELF)

La memoria del programa compilado se organiza en secciones estandarizadas:

Sección	Contenido	Ubicación Física	Propiedades
.isr_vector	Tabla de vectores de interrupción (punteros a handlers)	Inicio de Flash (0x0800_0000)	Solo lectura (r-x)
.text	Código ejecutable de las funciones en lenguaje máquina	Memoria Flash	Solo lectura y ejecución (r-x)
.rodata	Constantes (const), cadenas de texto literales ("Hola")	Memoria Flash	Solo lectura (r-)
.data	Variables globales y estáticas con valor inicial distinto de cero	LMA: Flash / VMA: RAM	Lectura y escritura (rw-)
.bss	Variables globales y estáticas no inicializadas o inicializadas a 0	Memoria RAM (0x2000_0000)	Lectura y escritura (rw-)
Stack	Variables locales, argumentos de funciones, registros apilados	Cima de la RAM (crece hacia abajo)	Lectura y escritura (rw-)
Heap	Memoria dinámica asignada por el usuario (malloc, FreeRTOS)	Base de RAM libre (crece hacia arriba)	Lectura y escritura (rw-)

3.3 Dualidad LMA vs VMA para la Sección .data

Un microcontrolador almacena el programa en memoria no volátil (Flash) pero ejecuta operando sobre variables en memoria volátil (RAM). Esto genera una dualidad fundamental de direcciones:

Definición de LMA vs VMA

- **LMA (Load Memory Address):** Dirección física donde reside la sección al momento de grabar (*flashear*) el chip. Es una dirección no volátil dentro de la memoria Flash.
- **VMA (Virtual / Execution Memory Address):** Dirección física donde la CPU accede y modifica los datos durante la ejecución normal del programa. Es una dirección dentro de la memoria RAM.

Para la sección .data: Sus valores iniciales (ej. `int sensor_offset = 120;`) deben guardarse en Flash (LMA) para no perderse ante cortes de energía, pero el procesador debe poder modificar su valor en RAM (VMA).

Sintaxis en el Linker Script (.ld):

```
.data :
{
    _sdata = .;           /* Marca de inicio de .data en RAM (VMA) */
    *(.data*)
    _edata = .;           /* Marca de fin de .data en RAM (VMA) */
} > RAM AT > FLASH       /* VMA en RAM, LMA en FLASH */
_sidata = LOADADDR(.data); /* Obtiene la dirección LMA de inicio en Flash */
```

3.4 El Linker Script (.ld) en Detalle

El Linker Script le indica al enlazador la topología exacta de la memoria física del microcontrolador mediante dos bloques principales:

1. **Bloque MEMORY:** Define los nombres, atributos de acceso, direcciones base y tamaños de las memorias físicas:

```
MEMORY
{
    FLASH (rx) : ORIGIN = 0x08000000, LENGTH = 64K
    RAM (rwx) : ORIGIN = 0x20000000, LENGTH = 20K
}
```

2. **Bloque SECTIONS:** Define cómo se distribuyen las secciones de los archivos objeto (.text, .rodata, .data, .bss) dentro de los bloques FLASH y RAM.

3.5 El Código de Arranque (Startup / Reset_Handler)

Al energizar el microcontrolador o presionar el botón de Reset, el hardware lee de la dirección 0x0800_0000 el valor del Stack Pointer inicial y salta a la dirección apuntada por el segundo vector: la función `Reset_Handler`.

Las 5 Responsabilidades Secuenciales del Startup (Reset_Handler)

Antes de transferir el control a la función `main()`, el código de arranque ejecuta estrictamente:

1. **Instanciar la Tabla de Vectores de Excepción:** Coloca en la dirección base de Flash (0x0800_0000) el valor del puntero `_estack` (cima de la RAM) y los punteros a todas las ISRs.
2. **Copiar la sección .data de Flash a RAM:** Lee byte a byte desde la dirección LMA (`_sidata` en Flash) y los escribe en el rango VMA en RAM (`_sdata` a `_edata`).
3. **Limpiar a Cero la sección .bss en RAM (Zero-Fill):** Escribe ceros (0x00) en todo el rango de memoria RAM delimitado entre los símbolos `_sbss` y `_ebss`.
4. **Llamar a la función de Inicialización de Hardware (SystemInit()):** Configura los osciladores HSE, activa el multiplicador PLL a 72 MHz, ajusta los prescalers de buses y configura los 2 wait states de la memoria Flash.
5. **Saltar a la función principal (main()):** Realiza el salto definitivo (`b1 main`). Si `main()` alguna vez retorna, entra en un bucle infinito de seguridad (`while(1);`).

4 Módulo 4: Controlador de Interrupciones y Excepciones (NVIC)

4.1 Concepto y Clasificación de Excepciones

Una **Excepción** es cualquier evento que altera el flujo secuencial normal de instrucciones de la CPU. En la arquitectura **ARMv7-M**, las interrupciones de periféricos son un subconjunto de las excepciones del sistema.

Excepción / IRQ	Posición	Offset	Prioridad	Descripción y Causa de Disparo
Initial SP	0	0x00	–	Valor cargado en MSP al resetear (<code>_estack</code>).
Reset	1	0x04	-3 (Fija)	Encendido (<i>Power-On</i>) o botón de Reset. Salta a <code>Reset_Handler</code> .
NMI	2	0x08	-2 (Fija)	Non-Maskable Interrupt. Eventos críticos (CSS: fallo de oscilador).
HardFault	3	0x0C	-1 (Fija)	Fallo genérico e irrecuperable de hardware o escalamiento de otro fallo.
MemManage	4	0x10	Programable	Violación de permisos de memoria definidos por la MPU.
BusFault	5	0x14	Programable	Error en el bus de datos o instrucciones (ej. acceso a dirección inválida).
UsageFault	6	0x18	Programable	Intento de ejecutar instrucción inválida, división por cero o $T = 0$.
SVCall	11	0x2C	Programable	Supervisor Call. Disparada por la instrucción SVC (usada por RTOS).
PendSV	14	0x38	Programable	Pendable Service Call. Excepción para cambio de contexto en RTOS.
SysTick	15	0x3C	Programable	Interrupción periódica del temporizador del núcleo (Tick del kernel).
IRQ0 - IRQ67	16 a 83	0x40+	Programable	Interrupciones de periféricos externos (EXTI, ADC, TIM, USART, etc.).

4.2 La Tabla de Vectores y Reubicación Dinámica (VTOR)

La tabla de vectores es un arreglo de punteros de 32 bits a funciones (*ISR Handlers*).

- Por defecto, tras el Reset reside en la dirección 0x0000_0000 (o 0x0800_0000 en Flash).
- **Bit 0 de cada puntero en 1:** Dado que Cortex-M3 solo soporta el juego Thumb, la dirección de cada Handler debe tener su bit menos significativo en 1 para que el salto no apague el bit T del registro EPSR.
- **Reubicación con VTOR (SCB→VTOR):** El registro *Vector Table Offset Register* permite reubicar la tabla a cualquier dirección alineada (por ejemplo a la RAM en 0x2000_0000). Esto es fundamental en **Bootloaders** que cargan aplicaciones nuevas en memoria.

4.3 Esquema de Prioridades del NVIC: Grupo vs Sub-prioridad

Cortex-M3 define registros de prioridad de 8 bits para cada interrupción, pero los fabricantes implementan solo los bits más significativos. El STM32F103 implementa los ****4 bits superiores**** ([7:4]), ofreciendo ****16 niveles de prioridad programables (0 a 15)****, donde el valor numérico ****0 representa la MÁXIMA prioridad**** y **15 la mínima**.

Preemption Priority (Grupo) vs Sub-prioridad (PRIGROUP)

El campo de 4 bits se subdivide mediante el registro de configuración SCB→AICR (campo PRIGROUP) en dos partes:

1. **Pre-emption Priority (Prioridad de Grupo / Desalojo):** Define si una interrupción entrante puede ****interrumpir y desalojar (anidarse)**** a una ISR que ya se está ejecutando en la CPU. Una interrupción solo puede desalojar a otra si su número de Pre-emption Priority es estrictamente MENOR.
2. **Sub-priority (Sub-prioridad):** Define el orden de atención cuando dos interrupciones del mismo grupo se disparan ****simultáneamente****. Una menor subprioridad NO puede desalojar a una ISR que ya comenzó a ejecutarse del mismo grupo.

4.4 Mecanismo de Auto-Stacking por Hardware y EXC_RETURN

A diferencia de arquitecturas tradicionales donde el programador debe salvar registros en ensamblador con instrucciones PUSH y POP, el Cortex-M3 realiza el apilamiento de forma ****totalmente automática por hardware****:

Paso	Acción del Hardware	Detalle Técnico
1. Entrada a Excepción	Auto-Stacking de 8 Registros (32 bytes)	El hardware empuja en el Stack activo (MSP o PSP): R0, R1, R2, R3, R12, LR, PC, xPSR en exactamente 12 ciclos de reloj. Son los registros <i>caller-saved</i> según el estándar AAPCS de ARM.
2. Vector Fetch en Paralelo	Búsqueda del Handler	Mientras el bus de datos apila los registros, el bus de instrucciones lee la dirección del handler de la Tabla de Vectores.
3. Carga de EXC_RETURN	Carga en el registro LR	El hardware sobrescribe el registro LR con un valor especial mágico con patrón 0xFFFFFFFx.
4. Retorno de Excepción	Instrucción BX LR	Al finalizar la ISR, la instrucción <code>bx lr</code> intenta saltar a la dirección 0xFFFFFFFx. El hardware intercepta este patrón y dispara la desapilación automática de los 8 registros restaurando el contexto previo.

Valores Clave de EXC_RETURN

- 0xFFFFFFF1: Retorno a **Handler Mode**, utilizando el **MSP**. (Ocurre en interrupciones anidadas).
- 0xFFFFFFF9: Retorno a **Thread Mode**, utilizando el **MSP**. (Común en programas Bare-Metal sin RTOS).
- 0xFFFFFFFD: Retorno a **Thread Mode**, utilizando el **PSP**. (Común al retornar a una tarea de usuario en un RTOS).

4.5 Optimizaciones de Latencia del NVIC

1. **Tail-Chaining (Encadenamiento de Cola):** Si al finalizar una ISR existe otra interrupción pendiente de igual o menor prioridad, la CPU ****NO desapila los 8 registros para volverlos a apilar****. Salta directamente al nuevo handler en solo ****6 ciclos de reloj**** (ahorrando más de 24 ciclos).

2. **Late Arrival (Llegada Tardía):** Si durante la secuencia de apilamiento para una interrupción de baja prioridad se dispara una de mayor prioridad, la CPU completa el apilamiento y atiende directamente a la de mayor prioridad sin reiniciar el proceso.
3. **Pop Preemption (Desalojo en Desapilamiento):** Si durante la secuencia de desapilamiento de retorno llega una interrupción de mayor prioridad, el hardware aborta el desapilamiento y transfiere el control al nuevo handler de inmediato.

4.6 Registros de Enmascaramiento Global del Núcleo

- **PRIMASK (1 bit):** Al ponerse en 1 (cpsid i), eleva el nivel de prioridad de ejecución de la CPU a 0. Deshabilita todas las interrupciones configurables, permitiendo únicamente NMI y HardFault.
- **FAULTMASK (1 bit):** Al ponerse en 1 (cpsid f), eleva el nivel de prioridad a -1. Deshabilita todo excepto NMI.
- **BASEPRI (8 bits):** Registro de enmascaramiento por umbral. Deshabilita todas las interrupciones con prioridad numéricamente igual o mayor a su valor (menor prioridad), permitiendo que interrupciones críticas sigan ejecutándose. Es el mecanismo utilizado por FreeRTOS para sus **Secciones Críticas** (configMAX_SYSCALL_INTERRUPT_PRIORITY).

4.7 El Controlador de Interrupciones Externas (EXTI) y Mapeo AFIO

El STM32 posee 16 líneas de interrupción externa independientes (EXTI0 a EXTI15) conectadas a los pines GPIO:

- Cada línea EXTI n puede conectarse al pin n de cualquiera de los puertos (PA n , PB n , PC n , etc.) mediante los registros AFIO->EXTICR[0..3].
- **Restricción:** No se pueden usar simultáneamente dos pines con el mismo número (ej. PA0 y PB0 no pueden generar EXTI0 a la vez).
- **Registros EXTI:**
 - EXTI->IMR: Interrupt Mask Register (1 = desmascara/habilita la interrupción).
 - EXTI->RTSR: Rising Trigger Selection (1 = habilita disparo por flanco ascendente).
 - EXTI->FTSR: Falling Trigger Selection (1 = habilita disparo por flanco descendente).
 - EXTI->PR: Pending Register (1 = interrupción ocurrida. **Se limpia escribiendo un 1**).

5 Módulo 5: Capas de Activación y Protocolos de Comunicación Serie

5.1 Las 4 Capas Jerárquicas de Activación de Periféricos en STM32

En microcontroladores modernos basados en ARM, los periféricos se encuentran completamente desenergizados e incommunicados al encender el chip. Su activación requiere una secuencia jerárquica de 4 capas:

Capa	Nombre y Función	Registros Involucrados
Capa 1: Infraestructura	Habilita la señal de reloj del bus correspondiente.	RCC->APB2ENR (Rápidos: GPIO, ADC1, USART1, SPI1) RCC->APB1ENR (Lentos: TIM2-4, USART2-3, SPI2, I2C1-2)
Capa 2: Interfaz Física	Configura los pines físicos (Entrada, Salida, AF, Analógico).	GPIOx->CRL, GPIOx->CRH, AFIO->EXTICR
Capa 3: Protocolo	Ajusta parámetros de comunicación (baud rate, paridad, polaridad).	USART->BRR, SPI->CR1, I2C->CCR, TIM->PSC/ARR
Capa 4: Operativa / IRQ	Enciende el módulo periférico y habilita interrupciones en el NVIC.	Bits UE, SPE, PE, ADON, CEN y función NVIC_EnableIRQ()

5.2 Sistema de Relojes (RCC) y el Clock Tree del STM32F103

El árbol de reloj del microcontrolador permite derivar frecuencias de trabajo estables para la CPU y los periféricos:

- **Fuentes Primarias de Oscilador:**
 - **HSI (High Speed Internal):** Oscilador interno RC de 8 MHz (arranque rápido, menor precisión térmica).
 - **HSE (High Speed External):** Cristal de cuarzo externo de 8 MHz (alta precisión y estabilidad).
 - **LSI (Low Speed Internal):** 40 kHz para el Watchdog independiente (IWDG) y RTC de bajo costo.
 - **LSE (Low Speed External):** Cristal de 32.768 kHz para el Real-Time Clock (RTC).
- **Multiplicador PLL (Phase-Locked Loop):** Multiplica la frecuencia de HSE (8 MHz) por $\times 9$ para generar la frecuencia máxima del sistema: $\text{SYSCLK} = 72 \text{ MHz}$.
- **Jerarquía de Buses y Prescalers:**
 - **AHB (Advanced High-performance Bus):** $\text{SYSCLK}/1 = 72 \text{ MHz}$. Alimenta la CPU, DMA y memoria.
 - **APB2 (Advanced Peripheral Bus 2 - Rápido):** $\text{HCLK}/1 = 72 \text{ MHz}$.
 - **APB1 (Advanced Peripheral Bus 1 - Lento):** $\text{HCLK}/2 = 36 \text{ MHz}$ (límite máximo de APB1).
 - **Reloj de Temporizadores en APB1:** Si el prescaler de APB1 es mayor a 1, la arquitectura duplica internamente la frecuencia para los timers ($\times 2$), por lo que **TIM2, TIM3 y TIM4 operan a 72 MHz**.
- **Latencia de Memoria Flash (Wait States):** La memoria Flash física tiene un tiempo de acceso de $\sim 40 \text{ ns}$. Para frecuencias $\text{SYSCLK} > 48 \text{ MHz}$ se deben configurar obligatoriamente ****2 ciclos de espera (LATENCY_2)**** y el buffer de prebúsqueda (PRFTBE) en el registro FLASH->ACR.

5.3 Capa GPIO: Decodificación de CNF y MODE

Cada pin se configura mediante 4 bits (CNF[1:0] y MODE[1:0]) en los registros CRL (pines 0 a 7) y CRH (pines 8 a 15):

CNF[1:0]	MODE[1:0]	Hex	Modo de Operación	Aplicación Típica
00	00	0x0	Entrada Analógica	Pines conectados al ADC (ej. PA4 potenciómetro)
01	00	0x4	Entrada Flotante (Reset default)	Pines RX de UART o entradas con circuito externo
10	00	0x8	Entrada con Pull-Up / Pull-Down	Botones y pulsadores (ODR=1 Pull-Up, ODR=0 Pull-Down)
00	01	0x1	Salida Push-Pull (10 MHz máx)	Control de LEDs y relés (baja velocidad, bajo ruido)
00	10	0x2	Salida Push-Pull (2 MHz máx)	LED integrado PC13
00	11	0x3	Salida Push-Pull (50 MHz máx)	Salidas digitales de alta velocidad
01	11	0x7	Salida Open-Drain (50 MHz máx)	Buses compartidos con pull-up (I2C)
10	11	0xB	Salida Función Alternativa Push-Pull (50 MHz)	Transmisor USART1 TX (PA9), TIM2 PWM (PA2), SPI1 MOSI
11	11	0xF	Salida Función Alternativa Open-Drain (50 MHz)	Líneas I2C1 (SCL en PB6, SDA en PB7)

5.4 Protocolo SPI (Serial Peripheral Interface)

SPI es un estándar de bus **serie síncrono, maestro-esclavo y full-duplex de alta velocidad** desarrollado por Motorola.

Línea	Nombre Completo	Función y Dirección de la Señal
MOSI	Master Out Slave In	Datos transmitidos del Maestro hacia el Esclavo.
MISO	Master In Slave Out	Datos transmitidos del Esclavo hacia el Maestro.
SCK / SCLK	Serial Clock	Señal de reloj generada exclusivamente por el Maestro para sincronizar los bits.
NSS / CS	Chip Select (Slave Select)	Selecciona al esclavo activo. Típicamente activo en nivel bajo (0V) .

Los 4 Modos de Reloj en SPI (CPOL y CPHA)

El protocolo SPI se parametriza mediante dos bits en el registro de control (SPI_CR1):

- **CPOL (Clock Polarity):** Define el estado de la línea SCK en reposo (*Idle*):
 - CPOL = 0 → SCK reposa en nivel ****BAJO (0V)****.
 - CPOL = 1 → SCK reposa en nivel ****ALTO (3.3V)****.
- **CPHA (Clock Phase):** Define en qué flanco se realiza el muestreo del dato:
 - CPHA = 0 → El dato se **muestrea en el 1er flanco** de reloj y **cambia** en el **2do flanco**.
 - CPHA = 1 → El dato **cambia en el 1er flanco** y se **muestrea en el 2do flanco**.

Modo SPI	CPOL	CPHA	Nivel de Reposo	Flanco de Muestreo (Captura de Dato)
Modo 0	0	0	Bajo (0V)	Primer Flanco (Ascendente / Flanco de Subida)
Modo 1	0	1	Bajo (0V)	Segundo Flanco (Descendente / Flanco de Bajada)
Modo 2	1	0	Alto (3.3V)	Primer Flanco (Descendente / Flanco de Bajada)
Modo 3	1	1	Alto (3.3V)	Segundo Flanco (Ascendente / Flanco de Subida)

Topologías de Red SPI: Estrella vs Daisy-Chain

1. **Topología Estrella (Esclavos Independientes):** Las líneas MOSI, MISO y SCK se comparten en paralelo. El Maestro posee una línea dedicada CS_k para cada esclavo. Para hablar con el esclavo k , el maestro baja CS_k = 0 y mantiene los demás en 1.
 - **Ventajas:** Protocolo simple, acceso aleatorio rápido a cualquier esclavo.
 - **Desventajas:** Consume un pin GPIO del maestro por cada esclavo añadido.
2. **Topología Daisy-Chain (Cadena en Cascada):** Se utiliza una sola línea CS común. El pin MOSI del maestro entra al MOSI del Esclavo 1; el MISO del Esclavo 1 entra al MOSI del Esclavo 2; y el MISO del último esclavo vuelve al MISO del maestro.
 - **Ventajas:** Requiere únicamente 4 pines en el microcontrolador sin importar la cantidad de esclavos.
 - **Desventajas:** Para actualizar un dispositivo hay que desplazar los datos a través de toda la cadena (comunicación serial secuencial rígida).

5.5 Protocolo I2C (Inter-Integrated Circuit)

I2C es un estándar de bus **serie síncrono, half-duplex y multimaestro** desarrollado por Philips (NXP) que utiliza únicamente ****2 líneas****:

Línea	Nombre	Descripción y Conexión Eléctrica
SDA	Serial Data	Línea bidireccional de datos en Drenador Abierto (<i>Open-Drain</i>).
SCL	Serial Clock	Línea de sincronización de reloj en Drenador Abierto con resistencia Pull-Up.

El Bus Open-Drain con Pull-Up y la Lógica Wired-AND

Ningún nodo del bus I2C puede forzar activamente un nivel alto (3.3V). Los transistores internos solo pueden drenar la línea a masa (0V) o quedar en alta impedancia (Hi-Z). El nivel alto lo imponen pasivamente resistencias de Pull-Up externas (2,2 kΩ a 4,7 kΩ).

- **Lógica Wired-AND:** Si cualquier nodo pone la línea en 0V, el bus entero pasa a 0V. Solo si TODOS los nodos están en Hi-Z, la línea sube a 3.3V.
- **Ventajas:** Inmune a cortocircuitos si dos transmisores emiten simultáneamente, y permite el mecanismo de Arbitraje Multimaestro y Clock Stretching.

Condiciones de Bus, Regla de Validez y Trama I2C

1. **Estado de Reposo (Idle):** Tanto SDA como SCL están en nivel ALTO (3.3V).
2. **Condición de START (S):** Transición de ALTO a BAJO en SDA mientras SCL está en ALTO. Inicia la comunicación.
3. **Condición de STOP (P):** Transición de BAJO a ALTO en SDA mientras SCL está en ALTO. Libera el bus.
4. **Regla de Validez del Dato:** Durante la transmisión de bits normales, SDA debe permanecer ESTABLE durante todo el semiperíodo de SCL en ALTO. La línea SDA SOLO puede cambiar de estado cuando SCL está en BAJO.
5. **Bit de ACK / NACK (Acknowledge):** Tras cada byte de 8 bits, en el 9no ciclo de reloj de SCL el receptor debe responder:
 - **ACK (0 lógico):** El receptor drena SDA a 0V indicando recepción correcta.
 - **NACK (1 lógico):** SDA queda en alto (el esclavo no existe, está ocupado o el maestro terminó de leer).
6. **Formato de Trama (Direccionamiento de 7 bits):**

START → [Dirección Esclavo (7 bits) + Bit $R/\bar{W}$ (1 bit)] → ACK → [Dato (8 bits)] → ACK → STOP

- $R/\bar{W} = 0$: El Maestro escribe datos en el Esclavo (Master Write).
- $R/\bar{W} = 1$: El Maestro lee datos desde el Esclavo (Master Read).

Características Avanzadas de I2C

- **Clock Stretching (Estiramiento de Reloj):** Si un esclavo lento necesita tiempo para procesar un byte recibido o preparar el siguiente, fuerza manualmente la línea SCL a nivel BAJO. El maestro detecta que SCL no sube y detiene su reloj hasta que el esclavo lo libera.
- **Arbitraje Multimaestro:** Cuando dos maestros inician una transmisión simultáneamente, comparan el bit que intentan enviar con el estado real de la línea SDA. Debido al Wired-AND, si un maestro intenta enviar un 1 pero detecta un 0 (porque otro maestro envió 0), **pierde el arbitraje de inmediato**, desconecta su transmisor y pasa a modo esclavo sin corromper la trama del ganador.
- **Modos de Velocidad:** Standard (100 kbps), Fast (400 kbps), Fast Plus (1 Mbps), High-Speed (3.4 Mbps).

5.6 Protocolo UART / USART y Estándar RS-232

UART (Universal Asynchronous Receiver-Transmitter) es un protocolo de comunicación serie asíncrono, full-duplex y punto a punto.

Línea	Nombre	Función y Niveles de Voltaje
TX	Transmit Data	Línea de salida de datos del transmisor.
RX	Receive Data	Línea de entrada de datos del receptor.
RTS / CTS	Hardware Flow Control	Request To Send (avisa que está listo) / Clear To Send (permiso para emitir).

Niveles TTL/CMOS vs Estándar RS-232

- **Nivel Lógico UART (Microcontrolador):** Opera entre 0V (0 lógico) y 3,3V o 5V (1 lógico). Apto únicamente para distancias de pocos centímetros sobre circuito impreso.
- **Nivel Eléctrico RS-232 (Industrial / PC):** Protocolo bipolar con lógica invertida para mayor inmunidad al ruido y alcance hasta 15-30 metros:
 - **1 lógico (Mark / Reposo):** Voltaje negativo entre -3V y -15V (típicamente -9V).
 - **0 lógico (Space / Start):** Voltaje positivo entre +3V y +15V (típicamente +9V).
- **Circuito de Adaptación:** Requiere un integrado conversor con bomba de carga como el MAX232 o MAX3232 que eleva e invierte los niveles de 3,3V a ±9V.

Estructura de la Trama Asíncrona (8N1)

Al no existir línea de reloj compartida, emisor y receptor deben acordar previamente una velocidad idéntica (Baud Rate).

Composición de un Carácter 8N1 (10 bits totales):

1. **Estado de Reposo (Mark):** Línea en nivel ALTO constante (1 lógico / -9V).
2. **Bit de START (1 bit):** Transición de ALTO a BAJO (0 lógico / +9V). Rompe el reposo y sincroniza los osciladores

internos del receptor.

3. **Bits de Datos (8 bits)**: Se transmiten siempre con el ****bit menos significativo (LSB) primero**** (bits D0 a D7).
 4. **Bit de Paridad (Opcional, 0 bits en 8N1)**: None, Even (Par: agrega un bit para que la suma de unos sea par) o Odd (Impar).
 5. **Bits de STOP (1 o 2 bits)**: Nivel ALTO (1 lógico) para delimitar el fin del carácter y permitir el próximo flanco de bajada.
- Eficiencia*: En 9600 baudios 8N1, la tasa útil es $\frac{9600 \text{ bps}}{10 \text{ bits/char}} = 960$ caracteres por segundo.

Sobremuestreo por 16 y Cálculo de BRR (Baud Rate Register)

El receptor de hardware del STM32 ejecuta un reloj interno a $16 \times \text{BaudRate}$:

1. Al detectar el flanco de bajada del Start bit, cuenta 8 pulsos de reloj para posicionarse exactamente en el ****centro geométrico del bit****.
2. Toma las muestras ****8, 9 y 10**** de cada bit y aplica un circuito de ****votación por mayoría de 2 sobre 3**** para rechazar ruidos transitorios.

Cálculo del Divisor de Baudios (USART_BRR)

$$\text{USARTDIV} = \frac{f_{\text{PCLK}}}{16 \times \text{BaudRate}}$$

- **Mantisa (bits [15:4])**: Parte entera de USARTDIV.
- **Fracción (bits [3:0])**: $\text{round}(\text{Parte Decimal} \times 16)$.

Ejemplo 1: 9600 bps con USART1 en APB2 (72 MHz):

$$\text{USARTDIV} = \frac{72,000,000}{16 \times 9600} = 468,75$$

- Mantisa = 468 = 0x1D4
- Fracción = $0,75 \times 16 = 12 = 0xC$
- USART1->BRR = 0x1D4C

Ejemplo 2: 115200 bps con USART1 en APB2 (72 MHz):

$$\text{USARTDIV} = \frac{72,000,000}{16 \times 115200} = 39,0625$$

- Mantisa = 39 = 0x27
- Fracción = $0,0625 \times 16 = 1 = 0x1$
- USART1->BRR = 0x0271

5.7 Gran Cuadro Comparativo de Protocolos Serie: SPI vs I2C vs UART

Parámetro	SPI	I2C	UART / USART
Tipo de Sincronismo	Síncrono (SCK explícito)	Síncrono (SCL explícito)	Asíncrono (acuerdo de baud rate)
Líneas Físicas	4 (MOSI, MISO, SCK, CS)	2 (SDA, SCL)	2 (TX, RX)
Modo de Transmisión	Full-Duplex	Half-Duplex	Full-Duplex
Topología	Maestro-Esclavo (Punto a Punto o Estrella con CS)	Multimaestro / Multiesclavo (Bus compartido)	Punto a Punto (1 a 1)
Direccionamiento	Físico por hardware (línea CS)	Por software (7 o 10 bits en trama)	Sin direccionamiento de bus
Nivel Eléctrico	Push-Pull (3.3V / 5V)	Open-Drain con Pull-Up externo	TTL (0-3.3V) o RS-232 ($\pm 9V$)
Velocidad Típica	10 a 50+ Mbps (Muy alta)	100 kbps, 400 kbps, hasta 3.4 Mbps	9600 bps a 115200+ bps (Lenta)
Detección de Errores	Ninguna por defecto en hardware	Bit de ACK/NACK en cada byte	Bit de Paridad opcional y flags FE/PE
Ventaja Principal	Máxima velocidad y simplicidad de hardware	Mínima cantidad de pines (solo 2 para decenas de sensores)	Universal en PCs, terminales y enlaces de larga distancia
Desventaja Principal	Requiere muchos pines a medida que crecen los esclavos	Más lento, sensible a capacitancia de bus y pull-ups	Sensible a diferencias de frecuencia de reloj entre nodos

6 Módulo 6: Sistemas Operativos en Tiempo Real (FreeRTOS)

6.1 Paradigma Super-Loop (Bare-Metal) vs RTOS

En sistemas embebidos existen dos enfoques arquitectónicos fundamentales:

1. **El Paradigma Super-Loop (Bucle Infinito Tradicional)**: Consiste en una función principal `while(1)` que ejecuta funciones de forma secuencial y atiende eventos mediante flags activadas en interrupciones (ISRs).
 - **Reglas de Oro del Super-Loop**:
 - a) **No ser Bloqueantes**: Ninguna función puede contener bucles de espera activa (`while(!(SPI->SR & TXE));`) ni demoras artificiales (`delay_ms()`).
 - b) **Ejecución Ultrarrápida**: Cada función debe ejecutar una porción mínima de código y retornar de inmediato.
 - c) **Descomposición en Máquinas de Estado Finitas (FSM)**: Las tareas complejas deben dividirse en múltiples estados.

- d) **Reentrancia:** Las funciones compartidas entre el bucle principal y las ISRs deben ser puramente reentrantes (no usar variables estáticas globales sin protección).
- **Limitación Fatal:** A medida que el sistema crece, el tiempo de ciclo del bucle se vuelve altamente variable (**Jitter excesivo**), imposibilitando garantizar tiempos de respuesta deterministas a eventos concurrentes.
2. **El Paradigma RTOS (Real-Time Operating System):** Divide la aplicación en múltiples **Tareas (Tasks)** concurrentes e independientes. El núcleo (*Kernel*) incluye un **Planificador (Scheduler)** que decide qué tarea utiliza la CPU en cada instante en base a prioridades y eventos, garantizando determinismo temporal.

6.2 Conceptos de Tiempo Real: Hard vs Soft Real-Time

Hard Real-Time vs Soft Real-Time

Un sistema de Tiempo Real no es aquel que opera a máxima velocidad ("lo más rápido posible"), sino aquel cuyo ****comportamiento temporal es estrictamente determinista y predecible****:

- **Hard Real-Time (Tiempo Real Estricto):** El cumplimiento de los plazos temporales (*deadlines*) es una restricción funcional absoluta. Si una respuesta llega tarde (incluso por microsegundos), se considera un **fallo catastrófico del sistema**, con riesgo de pérdidas de vidas o daños severos (ej. disparo de Airbag en choque en < 20 ms, frenos ABS, control de vuelo de un misil, marcapasos).
- **Soft Real-Time (Tiempo Real Blando):** El incumplimiento ocasional de un plazo degrada la calidad del servicio percibida, pero el sistema continúa funcionando de forma segura sin colapsar (ej. actualización de pantalla LCD, streaming de audio/video, adquisición de datos climáticos).

6.3 Arquitectura, Tipos de Datos y Notación Húngara en FreeRTOS

FreeRTOS es altamente portable y define sus propios tipos de datos independientes del compilador:

- **TickType_t:** Conteo de interrupciones de reloj del kernel (típicamente entero de 32 bits sin signo).
- **BaseType_t:** Tipo de dato más eficiente y natural para la arquitectura del microcontrolador (en ARM Cortex-M3 es un `long / int32_t` de 32 bits con signo).

Notación Húngara de Convención en FreeRTOS

- **Prefijos en Variables:**
 - c: char (ej. cBuffer)
 - s: short (16 bits)
 - l: long (32 bits)
 - x: BaseType_t, estructuras y handles de FreeRTOS (ej. xQueueHandle)
 - u: unsigned (modificador sin signo, ej. uxPriority, ulTimerCount)
 - p: Puntero (ej. pxHigherPriorityTaskWoken, pvParameters)
- **Prefijos en Funciones:** Indican el tipo de retorno y el archivo fuente donde están definidas:
 - vTask...: Función que retorna void, definida en tasks.c.
 - xQueue...: Función que retorna un BaseType_t, definida en queue.c.
 - prv...: Función privada (*static*) dentro de su archivo fuente.
- **Prefijos en Macros:**
 - pd...: Definición genérica de FreeRTOS (ej. pdTRUE = 1, pdFALSE = 0, pdPASS = 1).
 - err...: Códigos de error (ej. errQUEUE_FULL = 0).
 - port...: Definiciones específicas de la arquitectura hardware (ej. portMAX_DELAY).

6.4 Gestión de Tareas (Task Management)

En FreeRTOS, una tarea es un hilo de ejecución independiente con su propia pila (*Stack*) y un bloque de control (*Task Control Block - TCB*) que almacena su estado y prioridad.

Anatomía y Creación de una Tarea

Una función de tarea debe tener el siguiente prototipo estricto y **NUNCA debe retornar** mediante la instrucción `return`:

```
void vMiTarea(void *pvParameters)
{
    // Inicializacion local de la tarea
    for(;;)
    {
        // Código de ejecucion periodica o dirigida por eventos
        vTaskDelay(pdMS_TO_TICKS(100)); // Cede la CPU bloqueandose 100ms
    }
    // Si alguna vez sale del bucle, debe autoeliminarse obligatoriamente:
    vTaskDelete(NULL);
}
```

Creación con la API `xTaskCreate()`:

```

BaseType_t xTaskCreate(
    TaskFunction_t pxTaskCode,           // Puntero a la funcion de la tarea (ej. vMiTarea)
    const char * const pcName,          // Nombre descriptivo de texto (para debug)
    const configSTACK_DEPTH_TYPE usStackDepth, // Tamano del Stack en PALABRAS de 32 bits (128 = 512 bytes)
    void * const pvParameters,          // Puntero generico de parametros pasado a la tarea
    UBaseType_t uxPriority,              // Prioridad (0 = Idle / minima, configMAX_PRIORITIES-1 = maxima)
    TaskHandle_t * const pxCreatedTask // Puntero para guardar el Handler de la tarea
);

```

Máquina de Estados de una Tarea (FSM de 4 Estados)

1. **Running (En Ejecución):** La tarea está siendo ejecutada actualmente por el núcleo de la CPU. En procesadores de un solo núcleo, ****solo una tarea**** puede estar en estado Running en cada instante.
2. **Ready (Lista):** Tareas que tienen todo lo necesario para ejecutarse pero están esperando que el Scheduler les asigne la CPU porque actualmente corre otra tarea de igual o mayor prioridad.
3. **Blocked (Bloqueada):** La tarea está esperando un evento temporal (ej. transcurso de un delay con `vTaskDelay()`) o un evento de sincronización (llegada de un dato a una Cola, liberación de un Semáforo o Mutex). ****NO consume absolutamente ningún ciclo de CPU.****
4. **Suspended (Suspendida):** La tarea fue detenida explícitamente mediante la función `vTaskSuspend()`. Solo puede reanudarse si otra tarea o ISR invoca `vTaskResume()`.

6.5 El Planificador (Scheduler) y Algoritmo de Selección

FreeRTOS utiliza un **Planificador Preemptivo por Prioridades con Time-Slicing**:

- **Preemptivo por Prioridades:** En todo momento, el Scheduler garantiza que la CPU ejecute la tarea de mayor prioridad que se encuentre en estado ****Ready****. Si una tarea de mayor prioridad se desbloquea, desaloja inmediatamente (*preempts*) a la tarea de menor prioridad en ejecución.
- **Time-Slicing (Round-Robin):** Si existen múltiples tareas en estado Ready con exactamente la misma prioridad, el kernel comparte el tiempo de CPU entre ellas asignando un *quantum* de tiempo (1 Tick de SysTick) a cada una alternativamente.
- **Inanición (Starvation):** Ocurre cuando una tarea de alta prioridad se mantiene en estado Ready de forma permanente (ej. bucle infinito sin funciones de bloqueo `vTaskDelay` o colas), impidiendo que las tareas de menor prioridad se ejecuten jamás.
- **Algoritmo Optimizado por Puerto (configUSE_PORT_OPTIMISED_TASK_SELECTION = 1):** En ARM Cortex-M3, el kernel utiliza la instrucción en lenguaje máquina ****CLZ (Count Leading Zeros)**** para evaluar el mapa de bits de prioridades en ****tiempo constante $O(1)$ (un solo ciclo de reloj)****, eliminando la necesidad de recorrer listas enlazadas en C.

6.6 Temporización: `vTaskDelay()` vs `vTaskDelayUntil()`

Característica	<code>vTaskDelay(xTicksToDelay)</code>	<code>vTaskDelayUntil(&xLastWakeTime, xPeriod)</code>
Tipo de Retardo	Relativo: El retardo de N ticks se cuenta a partir del instante en que se invoca la función.	Absoluto: Calcula el tiempo de desbloqueo respecto al inicio del período anterior.
Deriva Temporal (Drift)	Sí produce deriva acumulativa (<i>jitter</i>): El período real total es Tiempo de Ejecución + Delay, desfasándose en el tiempo.	NO produce deriva (<i>Drift-Free</i>): Garantiza una frecuencia de ejecución periódica fija y exacta.
Uso Recomendado	Tareas esporádicas, parpadeo simple de LEDs no críticos, timeouts de reintento.	Muestreo de sensores en lazos de control (<i>PID</i>), lectura de encoders, generación de señales periódicas.

6.7 La Tarea Idle y la Función Idle Hook

La tarea **Idle** es creada automáticamente por el kernel al iniciar el Scheduler (`vTaskStartScheduler()`) con la ****prioridad más baja posible (Prioridad 0)****.

1. Responsabilidades de la Tarea Idle:

- Garantizar que siempre exista al menos una tarea en estado Ready para que la CPU nunca quede flotante sin instrucciones que ejecutar.
 - ****Liberar la memoria dinámica**** (Stack y TCB asignados por el Heap) de aquellas tareas que hayan sido eliminadas mediante la función `vTaskDelete()`.
2. La Función `vApplicationIdleHook()`: Si la macro `configUSE_IDLE_HOOK` está en 1, el kernel invoca automáticamente esta función en cada ciclo de la tarea Idle.
- **Usos típicos:** Poner el microcontrolador en modo de bajo consumo (`__WFI()` - Wait For Interrupt) o refrescar el perro guardián (*Watchdog*).
 - **Regla de Oro Indispensable:** ****ESTÁ TERMINANTEMENTE PROHIBIDO** que `vApplicationIdleHook()` se bloquee, demore o invoque funciones con timeout (ej. `vTaskDelay()`), ya que **si la tarea Idle se bloquea, el kernel colapsa y no se puede liberar memoria**.

6.8 Mecanismos de Comunicación: Colas de Mensajes (Queues)

Las Colas son el mecanismo primario de comunicación segura (*Thread-Safe*) entre tareas y entre interrupciones y tareas.

Almacenamiento por "Copia por Valor" (Deep Copy)

FreeRTOS almacena los datos en las colas copiando la información byte a byte (*Pass by Value*) en lugar de solo copiar una referencia o puntero.

- **Ventajas:**

1. **Desacoplamiento Total:** El emisor puede reutilizar o sobrescribir su variable local inmediatamente después de llamar a `xQueueSend()` sin temor a corromper los datos del receptor.
2. **Seguridad con Variables Locales del Stack:** Permite enviar variables locales asignadas en la pila de una función sin riesgo de que la función retorne y libere la memoria antes de que el receptor la lea.

- **Paso de Punteros para Datos Grandes (> 64 bytes):** Si el dato es grande, copiar byte a byte consume mucho tiempo. En ese caso, se encola por valor el ****puntero a la estructura**** (`MiStruct_t *pMsg`).

- **Reglas de Seguridad Obligatorias al Enviar Punteros:**

1. **Transferencia Estricta de Propiedad (*Ownership Handover*):** Una vez enviado el puntero, el emisor tiene terminantemente prohibido volver a acceder o modificar el buffer.
2. **Memoria Dinámica Persistente:** El buffer apuntado debe residir en el Heap (`pvPortMalloc()`) o en variables globales, nunca en el stack local de una función efímera.
3. **Liberación de Memoria:** El receptor es el responsable exclusivo de liberar la memoria dinámica (`pvPortFree()`) una vez procesado el mensaje.

Funciones Principales de la API de Colas

- `QueueHandle_t xQueueCreate(uxQueueLength, uxItemSize)`: Crea una cola en el Heap para N elementos de tamaño M bytes.
- `BaseType_t xQueueSendToBack(xQueue, pvItem, xTicksToWait)`: Envía al final (FIFO). Si la cola está llena, la tarea se bloquea hasta `xTicksToWait` (usar `portMAX_DELAY` para esperar indefinidamente).
- `BaseType_t xQueueReceive(xQueue, pvBuffer, xTicksToWait)`: Lee y extrae el primer elemento de la cola. Si está vacía, la tarea se bloquea hasta `xTicksToWait`.
- `UBaseType_t uxQueueMessagesWaiting(xQueue)`: Retorna la cantidad de elementos sin leer.

6.9 Gestión de Interrupciones en FreeRTOS y Procesamiento Diferido**El API Seguro para Interrupciones (...FromISR) y `pxHigherPriorityTaskWoken`**

Las rutinas de servicio de interrupción (ISRs) de hardware ****NO** tienen contexto de tarea****** y por ende ****NUNCA** pueden bloquearse******. FreeRTOS proporciona una versión especial de sus funciones de API con el sufijo `...FromISR`.

El Parámetro `pxHigherPriorityTaskWoken`:

1. Se pasa la dirección de una variable inicializada en `pdFALSE`: `BaseType_t xHigherPriorityTaskWoken = pdFALSE;`
2. Si al enviar un dato a una cola o entregar un semáforo desde la ISR se desbloquea una tarea que tiene una prioridad mayor a la tarea que fue interrumpida, la función `...FromISR` cambia automáticamente la variable a `pdTRUE`.
3. Al final de la ISR, se invoca la macro `portYIELD_FROM_ISR(xHigherPriorityTaskWoken)`. Si la bandera es `pdTRUE`, el procesador solicita la excepción `PendSV` para que, al retornar de la ISR, la CPU ejecute de inmediato la tarea de alta prioridad recién desbloqueada sin esperar al próximo Tick del SysTick.

Patrón de Procesamiento Diferido (Deferred Interrupt Processing)

Problema: Mantener una ISR ejecutando código prolongado (formateo de texto, cálculos complejos, escrituras lentas) bloquea otras interrupciones, aumenta el jitter del sistema y degrada el determinismo.

Solución en Dos Mitades (Top-Half / Bottom-Half):

1. **Top-Half (La ISR de Hardware):** Es ultracorta (típicamente microsegundos). Lee el registro de datos del periférico, limpia la bandera de interrupción, entrega un **Semáforo Binario** mediante `xSemaphoreGiveFromISR()` y fuerza el cambio de contexto con `portYIELD_FROM_ISR()`.
2. **Bottom-Half (Tarea Servidora en FreeRTOS):** Una tarea de alta prioridad permanece bloqueada esperando el semáforo (`xSemaphoreTake(xSem, portMAX_DELAY)`). Al liberarse el semáforo, la tarea se despierta inmediatamente y ejecuta todo el procesamiento pesado de datos a nivel de hilo (Thread Mode), permitiendo que nuevas interrupciones de hardware se sigan atendiendo sin retrasos.

7 Módulo 7: Banco de 30 Preguntas Típicas de Examen Teórico con Respuestas Modelo

1

¿Qué establece la Ley de Moore y cuáles fueron sus consecuencias técnicas y económicas directas?

Respuesta: Enunciada por Gordon Moore en 1965, establece que *el número de transistores integrados en un circuito integrado se duplica aproximadamente cada 18 a 24 meses*.

Consecuencias Directas:

1. **Económica (Costo por Transistor):** Al duplicarse la densidad con costos de oblea similares, el costo por transistor cayó de forma exponencial, permitiendo la masificación de computadoras, teléfonos celulares y sistemas embebidos de bajo costo.
2. **Técnica (Velocidad y Consumo):** La reducción del tamaño de canal y de las capacidades parásitas permitió aumentar drásticamente la frecuencia de reloj y la integración de periféricos complejos (SoC).

2

¿Cuáles son las cuatro restricciones fundamentales de diseño que diferencian a los Sistemas Embebidos de las PCs y Servidores?

Respuesta:

1. **Costo Extremo:** Producidos en grandes volúmenes donde centavos de dólar deciden la viabilidad del producto.
2. **Consumo de Potencia y Energía:** Crítico para autonomía en baterías y para operar sin ventiladores mecánicos.
3. **Confiabilidad y Seguridad Funcional:** Un fallo puede ocasionar desastres económicos o pérdida de vidas humanas (aviónica, automotriz, médico).
4. **Determinismo Temporal (Real-Time):** Obligación de responder a eventos externos dentro de límites temporales estrictos garantizados (*deadlines*).

3

Explique la diferencia entre Kilobyte (KB) y Kibibyte (KiB) según la norma IEC 80000-13 y por qué es crítico en arquitectura de memoria.

Respuesta:

- **Kilobyte (KB):** Unidad decimal basada en potencias de 10 ($1 \text{ KB} = 10^3 = 1,000 \text{ bytes}$).
- **Kibibyte (KiB):** Unidad binaria formalizada por IEC ($1 \text{ KiB} = 2^{10} = 1,024 \text{ bytes}$).

Importancia en Arquitectura: El hardware digital decodifica memoria mediante líneas de dirección binarias (2^N). Un microcontrolador con bus de 16 líneas decodifica exactamente $2^{16} = 65,536 \text{ bytes} = 64 \text{ KiB}$, lo que difiere en un $+2,4\%$ respecto a 64,000 bytes (64 KB). A medida que la memoria crece (GiB vs GB), la discrepancia supera el 7 %, siendo crítico para dimensionar buffers y tablas de paginación.

4

Describa las 8 grandes ideas de la arquitectura de computadores según Patterson y Hennessy.

Respuesta:

1. **Diseño para la Ley de Moore:** Proyectar el diseño para la tecnología del año de fabricación.
2. **Uso de la Abstracción:** Jerarquía de capas para ocultar detalles de bajo nivel.
3. **Hacer el caso común rápido:** Optimizar las instrucciones de mayor frecuencia de uso.
4. **Rendimiento mediante Paralelismo:** Procesamiento simultáneo de datos e instrucciones.
5. **Rendimiento mediante Segmentación (Pipelining):** Solapar etapas de instrucciones.
6. **Rendimiento mediante Predicción:** Anticipar ramas y accesos a memoria.
7. **Jerarquía de Memorias:** Combinar memoria rápida pequeña con memoria lenta grande.
8. **Confiabilidad mediante Redundancia:** Tolerancia a fallos con componentes de respaldo.

5

¿Qué es una ISA (Instruction Set Architecture) y por qué garantiza la independencia de implementación?

Respuesta: La ISA es la interfaz abstracta que especifica las instrucciones de máquina, registros visibles, modos de direccionamiento y manejo de excepciones de un procesador.

Garantiza la independencia de implementación porque desacopla el qué (la especificación lógica) del cómo (la microarquitectura física). Permite que el mismo binario ejecute en silicios con microarquitecturas radicalmente distintas (ej. Cortex-M3 vs Cortex-M4 o procesadores multinúcleo) preservando la compatibilidad de software.

6

Deduzca la ecuación de tiempo de CPU y explique cómo impactan el Compilador, la ISA y la Tecnología de Silicio en sus tres variables.

Respuesta:

$$\text{Tiempo de CPU} = IC \times CPI \times T_{\text{clock}} = \frac{IC \times CPI}{f_{\text{clock}}}$$

- **Compilador:** Impacta en IC (generación eficiente de instrucciones) y en CPI (planificación de instrucciones para evitar bloqueos de pipeline).
- **ISA:** Impacta en IC (instrucciones potentes reducen IC) y en CPI (instrucciones RISC reducen CPI).

- **Tecnología de Silicio / Microarquitectura:** Define T_{clock} (frecuencia máxima f_{clock}) y la cantidad de etapas del pipeline que determinan el CPI base.

7

¿Qué es el "Muro de Potencia" (Power Wall), qué ecuación física lo describe y qué cambio de paradigma provocó en 2006?

Respuesta: Es el límite físico que impidió seguir aumentando la frecuencia de reloj en procesadores mononúcleo debido a la imposibilidad de disipar el calor generado.

Ecuación: $P_{\text{dinámica}} = \frac{1}{2} \cdot C \cdot V_{DD}^2 \cdot f \cdot \alpha$.

Al no poder reducir V_{DD} por debajo de $\sim 0,8\text{V}$ (debido al incremento exponencial de corrientes de fuga sub-umbral), aumentar la frecuencia f disparaba la potencia térmica por encima de los límites de enfriamiento ($> 100 \text{ W/cm}^2$). En 2006 forzó el cambio de paradigma hacia procesadores **Multicore** a frecuencias moderadas.

8

Compare la filosofía RISC vs CISC en relación al conjunto de instrucciones, formato y acceso a memoria.

Respuesta:

- **RISC:** Conjunto reducido de instrucciones de longitud fija (16/32 bits), formato regular, arquitectura Load/Store (solo LDR/STR tocan memoria, operaciones aritméticas solo en registros), CPI cercano a 1, optimización delegada al compilador.
- **CISC:** Conjunto amplio de instrucciones complejas de longitud variable (1 a 15 bytes), operaciones aritméticas directas sobre memoria, CPI variable y elevado, hardware de decodificación microprogramado complejo.

9

¿Cuál es la diferencia entre Arquitectura Mapeada en Memoria (Memory-Mapped I/O) y Mapeada en Puertos (Port-Mapped I/O)?

Respuesta:

- **Memory-Mapped I/O (ARM Cortex-M):** Los registros de periféricos comparten el mismo espacio de direccionamiento de 32 bits que la memoria RAM y Flash. Se accede a ellos con las mismas instrucciones estándar (LDR y STR).
- **Port-Mapped I/O (x86):** Los periféricos residen en un espacio de direcciones de E/S separado e independiente de la memoria principal, requiriendo instrucciones especiales de la CPU (como IN y OUT) y señales de control específicas en el bus.

10

¿Por qué los registros de los periféricos deben declararse obligatoriamente con el calificador 'volatile' en C?

Respuesta: Porque le indica al compilador que el valor de esa dirección física puede cambiar en cualquier instante por hardware sin intervención del software.

Si no se declara volatile, el optimizador del compilador asume que una variable que no es modificada explícitamente en un bucle mantiene su valor constante, guardándola en un registro interno de la CPU (caching) y suprimiendo las lecturas sucesivas a la memoria física, provocando bucles infinitos en esperas de banderas de periféricos.

11

¿Por qué el NVIC y el SysTick están ubicados en el Private Peripheral Bus (PPB) a partir de la dirección 0xE000_0000?

Respuesta:

1. **Baja Latencia y Determinismo:** El bus PPB conecta directamente el NVIC y SysTick con el núcleo de la CPU, sin competir con el tráfico de periféricos en buses AHB/APB.
2. **Seguridad y Privilegio:** El hardware bloquea accesos no privilegiados al PPB generando fallos de protección.
3. **Estandarización y Portabilidad CMSIS:** Al residir en direcciones idénticas en todos los microcontroladores ARM Cortex-M3 del mercado, el código de manejo de interrupciones y reloj de sistema es 100 % portable.

12

Explique el funcionamiento de la región Bit-Banding en Cortex-M3, sus ventajas y demuestre la fórmula de cálculo.

Respuesta: Mapea cada bit individual de una región de 1 MB (*Bit-Band Region*) a una palabra de 32 bits en otra región de 32 MB (*Bit-Band Alias*).

Ventaja: Permite modificar un único bit de un periférico o variable en RAM de forma totalmente **atómica** mediante una sola instrucción STR, eliminando condiciones de carrera con interrupciones sin tener que deshabilitar el NVIC.

Fórmula: $\text{AliasAddr} = \text{AliasBase} + (\text{ByteOffset} \times 32) + (\text{BitNumber} \times 4)$.

13

Describa las 4 fases de la cadena de construcción (Toolchain) de GNU GCC y qué produce cada una.

Respuesta:

1. **Preprocesador (cpp):** Procesa directivas `#include` y macros `#define` generando código C expandido (`.i`).
2. **Compilador (cc1):** Analiza la sintaxis, optimiza y genera código ensamblador (`.s`).
3. **Ensamblador (as):** Traduce en dos pasadas los mnemónicos a código máquina binario en formato objeto relocable ELF (`.o`).
4. **Enlazador (ld):** Combina archivos objeto con bibliotecas y el Linker Script (`.ld`), resuelve símbolos y emite el archivo ejecutable final (`.elf`).

14

Explique la dualidad LMA vs VMA para la sección `.data` en un sistema embebido y cómo se implementa en el Linker Script.

Respuesta: Las variables globales inicializadas necesitan que su valor inicial persista ante cortes de energía (deben residir en Flash - **LMA: Load Memory Address**), pero deben ser modificables durante la ejecución del programa (deben residir en RAM - **VMA: Virtual/Execution Memory Address**).

En el Linker Script se implementa con: `.data : { _sdata = .; *(.data*); _edata = .; } >RAM AT >FLASH`, exportando el símbolo `_sdata = LOADADDR(.data)`; para el código de inicio.

15

Detalle las 5 responsabilidades secuenciales que ejecuta el archivo de arranque (Startup / Reset_Handler) antes de saltar a `main()`.

Respuesta:

1. Instanciar la Tabla de Vectores definiendo el Stack Pointer inicial (`_estack`) y el puntero a `Reset_Handler`.
2. Copiar la sección `.data` desde Flash (LMA `_sdata`) hacia RAM (VMA `_sdata` a `_edata`).
3. Limpiar a cero la sección `.bss` en RAM (entre los símbolos `_sbss` y `_ebss`).
4. Llamar a la función de inicialización de relojes y memoria Flash (`SystemInit()`).
5. Saltar a la función principal del usuario (`main()`).

16

¿Qué diferencia existe entre Pre-emption Priority (Prioridad de Grupo) y Sub-prioridad en el NVIC de ARM Cortex-M?

Respuesta:

- **Pre-emption Priority (Prioridad de Grupo):** Define si una nueva interrupción puede interrumpir y desalojar (anidarse) a una ISR que ya se está ejecutando (solo desaloja si su número de grupo es estrictamente menor).
- **Sub-prioridad:** Solo actúa como criterio de desempate cuando dos interrupciones del mismo grupo se solicitan de forma simultánea. Una menor subprioridad ****NO desaloja**** a otra del mismo grupo que ya esté en ejecución.

17

¿Qué registros apila automáticamente el hardware del Cortex-M3 al entrar a una excepción y por qué se eligieron esos registros?

Respuesta: Apila automáticamente 8 registros (32 bytes) en el Stack activo: **R0, R1, R2, R3, R12, LR, PC y xPSR** en 12 ciclos de reloj.

Se eligieron porque según el estándar **AAPCS** de ARM, estos son los registros *caller-saved* (que cualquier función C estándar puede sobrescribir). Al apilarlos por hardware, las ISRs pueden escribirse como funciones ordinarias en C sin necesidad de prólogos o epílogos en ensamblador.

18

¿Qué es el valor **EXC_RETURN**, dónde se carga al entrar a una interrupción y qué sucede cuando la CPU intenta saltar a él al retornar?

Respuesta: Es un valor mágico con patrón `0xFFFFFFFx` que el hardware carga en el registro **LR** al ingresar a una excepción, codificando el modo previo y la pila utilizada.

Cuando la ISR finaliza con la instrucción `bx lr`, la CPU detecta la dirección `0xFFFFFFFx`, activa automáticamente la secuencia de retorno por hardware, desapila los 8 registros restaurando el contexto previo y reanuda la ejecución en el modo original (Thread o Handler).

19

Explique la diferencia de función entre los registros especiales PRIMASK y BASEPRI.

Respuesta:

- **PRIMASK (1 bit):** Al activarse (CPSID = 1) eleva la prioridad de la CPU a 0, bloqueando todas las interrupciones programables del sistema (solo permite NMI, Reset y HardFault).
- **BASEPRI (8 bits):** Permite un enmascaramiento selectivo por umbral: bloquea solo aquellas interrupciones con prioridad numéricamente igual o mayor a su valor (menor prioridad), permitiendo que interrupciones de tiempo real crítico sigan atendiéndose (usado en FreeRTOS para secciones críticas).

20

Describa las 4 capas jerárquicas para activar cualquier periférico en un microcontrolador STM32.

Respuesta:

1. **Capa de Infraestructura (RCC):** Habilitar el reloj en el bus correspondiente (APB1ENR o APB2ENR).
2. **Capa de Interfaz Física (GPIO/AFIO):** Configurar el modo de los pines asociados (Entrada, Salida, Función Alternativa).
3. **Capa de Protocolo:** Configurar baud rate, prescaler, paridad, polaridad y registros de control específicos.
4. **Capa Operativa / IRQ:** Activar el bit de encendido funcional del módulo (UE, SPE, PE, ADON, CEN) y habilitar la interrupción en el NVIC.

21

Explique el significado de los bits CPOL y CPHA en el protocolo SPI y deduzca en qué flanco se leen los datos en Modo 0 y Modo 3.

Respuesta:

- **CPOL (Polaridad):** Nivel de reposo de SCK (0 = Bajo/0V, 1 = Alto/3.3V).
- **CPHA (Fase):** Flanco de muestreo (0 = Primer flanco, 1 = Segundo flanco).

Deducción:

- **Modo 0 (CPOL=0, CPHA=0):** Reposa en bajo → El primer flanco es **Ascendente (Subida)**. El dato se muestrea en flanco ascendente.
- **Modo 3 (CPOL=1, CPHA=1):** Reposa en alto → El primer flanco es descendente, el segundo es **Ascendente (Subida)**. El dato se muestrea en flanco ascendente.

22

¿Por qué las líneas SDA y SCL del bus I2C operan en Drenador Abierto (Open-Drain) con resistencias Pull-Up y qué ventaja otorga en arbitraje?

Respuesta: Porque ningún nodo puede forzar un nivel alto activo (3.3V); solo pueden drenar a 0V o quedar en alta impedancia (Hi-Z), siendo el nivel alto provisto pasivamente por las resistencias Pull-Up externas.

Esto genera una configuración lógica **Wired-AND** que elimina cortocircuitos si múltiples nodos transmiten simultáneamente y permite el ****Arbitraje Multimaestro sin pérdida de datos****: el nodo que envía un 1 (Hi-Z) y detecta un 0 (otro nodo drenó la línea) detecta inmediatamente que perdió el arbitraje y se retira sin corromper el mensaje del ganador.

23

Defina las condiciones de START y STOP en el protocolo I2C y explique la regla de validez de datos.

Respuesta:

- **START (S):** Transición de ALTO a BAJO en SDA mientras SCL está en ALTO.
- **STOP (P):** Transición de BAJO a ALTO en SDA mientras SCL está en ALTO.
- **Regla de Validez de Datos:** Durante la transmisión de bits de datos, la línea SDA ****debe permanecer estable**** mientras SCL está en nivel ALTO. La línea SDA ****únicamente puede conmutar cuando SCL está en nivel BAJO****.

24

En una comunicación UART 9600 8N1, ¿cuántos caracteres útiles por segundo se transfieren y qué representa el bit de START?

Respuesta: Una trama 8N1 contiene 10 bits por carácter (1 Start + 8 Datos + 0 Paridad + 1 Stop).

Tasa de transferencia útil:

$$\text{Caracteres/segundo} = \frac{9600 \text{ bits/s}}{10 \text{ bits/carácter}} = 960 \text{ caracteres/s}$$

El bit de START es un nivel 0 lógico (flanco descendente) que rompe el estado de reposo en nivel alto de la línea y sincroniza los temporizadores de muestreo internos del receptor.

25

Calcule el valor a cargar en el registro `USART_BRR` para una velocidad de 115200 baudios con `USART1` en APB2 a 72 MHz.

Respuesta:

$$\text{USARTDIV} = \frac{f_{\text{PCLK2}}}{16 \times \text{BaudRate}} = \frac{72,000,000}{16 \times 115200} = 39,0625$$

1. **Parte Entera (Mantisa):** $39 = 0x027$ (se carga en bits [15:4]).
2. **Parte Fraccionaria:** $0,0625 \times 16 = 1 = 0x1$ (se carga en bits [3:0]).
3. **Valor Final en `USART1->BRR`:** $0x0271$

26

¿Cuál es la diferencia conceptual entre un sistema **Hard Real-Time** y **Soft Real-Time**? Dé un ejemplo de cada uno.

Respuesta:

- **Hard Real-Time:** El incumplimiento de un plazo temporal (*deadline*) es un fallo total y catastrófico con consecuencias destructivas o riesgo de vidas (ej. despliegue de Airbag automotriz en < 20 ms).
- **Soft Real-Time:** El incumplimiento del plazo degrada la calidad del servicio pero el sistema continúa operando de forma segura (ej. caída de cuadros por segundo en la reproducción de un video o actualización de UI).

27

Compare `vTaskDelay()` vs `vTaskDelayUntil()`. ¿Por qué `vTaskDelay()` produce deriva temporal (*drift/jitter*) en tareas periódicas?

Respuesta: `vTaskDelay()` introduce un retardo relativo a partir del instante en que es invocada, por lo que el período real es Tiempo de Ejecución de la Tarea + Delay, acumulando un corrimiento temporal inevitable (*drift*).

`vTaskDelayUntil()` calcula el retardo en base al tiempo del ciclo anterior de forma absoluta, garantizando una frecuencia de ejecución periódica fija y determinista sin deriva temporal.

28

¿Cuáles son las dos responsabilidades de la tarea `Idle` en FreeRTOS y cuál es la regla de oro de la función `Idle Hook`?

Respuesta: Responsabilidades de `Idle`:

1. Garantizar que siempre haya al menos una tarea en estado Ready para ejecutar en la CPU.
2. Liberar la memoria dinámica del Stack y TCB de las tareas eliminadas mediante `vTaskDelete()`.

Regla de Oro del `Idle Hook`: Está **terminantemente prohibido** que la función `vApplicationIdleHook()` contenga operaciones bloqueantes, esperas o funciones con delay, ya que si `Idle` se bloquea el sistema colapsa.

29

¿Por qué FreeRTOS implementa almacenamiento "Por Copia de Valor" en sus colas y qué precauciones deben tomarse al enviar punteros?

Respuesta: Implementa copia de valor (byte a byte) para desacoplar totalmente al emisor del receptor y permitir el pasaje seguro de variables locales del stack sin riesgo de corrupción al retornar la función.

Precauciones al enviar Punteros:

1. **Transferencia de Propiedad:** El emisor no debe volver a tocar la memoria apuntada tras encolar el puntero.
2. **Memoria Válida:** La memoria debe residir en el Heap o variables globales, nunca en stacks efímeros.
3. **Liberación:** El receptor es el responsable exclusivo de liberar la memoria dinámica (`vPortFree()`).

30

Explique el patrón de "Procesamiento Diferido de Interrupciones" en FreeRTOS utilizando semáforos binarios y `pxHigherPriorityTaskWoken`.

Respuesta: Consiste en dividir la atención del evento en dos etapas:

1. **La ISR de hardware (Top-Half):** Es ultracorta: atiende el hardware, limpia la bandera, desbloquea una tarea de alta prioridad con `xSemaphoreGiveFromISR(&xHigherPriorityTaskWoken)` y solicita el cambio de contexto inmediato con `portYIELD_FROM_ISR(xHigherPriorityTaskWoken)`.
2. **La Tarea Servidora (Bottom-Half):** Permanece bloqueada esperando el semáforo (`xSemaphoreTake(xSem, portMAX_DELAY)`). Al liberarse, se ejecuta inmediatamente y realiza el procesamiento pesado a nivel de hilo (Thread Mode), permitiendo que nuevas interrupciones de hardware se sigan atendiendo sin retrasos ni degradación del determinismo.

8 Módulo 8: Actividades y Ejercicios Extraídos de la Bibliografía Oficial

En esta sección se compilan los ejercicios y problemas de estudio extraídos directamente de los libros de texto de referencia utilizados por la cátedra en cada unidad temática.

8.1 Fuente 1: Patterson & Hennessy — *Computer Organization and Design (ARM Edition)*

Problema P&H 1.12: Falacias de Rendimiento y Frecuencia de Reloj (Capítulo 1)

Considere dos procesadores P_1 y P_2 ejecutando dos variantes del mismo algoritmo:

- **Procesador P_1 :** Frecuencia $f_1 = 4$ GHz, $CPI_1 = 0,9$, requiere ejecutar $IC_1 = 5,0 \times 10^9$ instrucciones.
 - **Procesador P_2 :** Frecuencia $f_2 = 3$ GHz, $CPI_2 = 0,75$, requiere ejecutar $IC_2 = 1,0 \times 10^9$ instrucciones.
- a) ¿Es verdadero que el procesador con mayor frecuencia de reloj (P_1) tiene siempre un mayor rendimiento? Calcule el tiempo de CPU para ambos.
 - b) Si un nuevo compilador optimiza el código de P_1 reduciendo sus instrucciones a $1,0 \times 10^9$ manteniendo su $CPI_1 = 0,9$, ¿cuál procesador es más rápido y por qué factor?

Problema P&H 1.14: Ecuación de Rendimiento y Mezcla de Instrucciones (Capítulo 1)

Un programa ejecuta la siguiente mezcla de instrucciones en un procesador con frecuencia de reloj $f = 2$ GHz:

- 50×10^6 instrucciones de Punto Flotante (FP) con $CPI_{FP} = 1$.
 - 110×10^6 instrucciones Enteras (INT) con $CPI_{INT} = 1$.
 - 80×10^6 instrucciones de Carga/Almacenamiento (L/S) con $CPI_{L/S} = 4$.
 - 16×10^6 instrucciones de Salto (Branch) con $CPI_{Branch} = 2$.
- a) Calcule el CPI ponderado promedio del programa y el tiempo total de ejecución de CPU.
 - b) ¿En cuánto debe mejorarse el CPI de las instrucciones FP si se desea que el programa ejecute 2 veces más rápido (reducción del 50 % en tiempo)? ¿Es físicamente posible?
 - c) ¿En cuánto debe mejorarse el CPI de las instrucciones L/S si se desea duplicar el rendimiento global del programa?
 - d) ¿En cuánto mejora el tiempo de ejecución si se optimiza el procesador reduciendo el CPI de INT y FP en un 40 % y el CPI de L/S y Branch en un 30 %?

Problema P&H 1.15: Ley de Amdahl y Overhead en Multiprocesamiento (Capítulo 1)

Un programa requiere $t = 100$ s de ejecución secuencial en 1 solo procesador. Al adaptarse para ejecutar en un sistema con p procesadores, el tiempo de cálculo paralelo se divide como t/p , pero se introducen 4 s fijos de sobrecarga (overhead) por sincronización y secciones críticas, independientemente del número de procesadores.

- a) Calcule el tiempo de ejecución y el factor de aceleración (S) para $p = 2, 4, 8, 16$ y 64 procesadores.
- b) ¿Cuál es el límite máximo teórico de aceleración que se puede alcanzar con infinitos procesadores ($p \rightarrow \infty$)?

Problema P&H 2.12: Flujo de Compilación y Resolución de Símbolos en Enlazado (Capítulo 2)

Explique cómo el enlazador (*Linker*) resuelve las referencias a funciones externas y constantes:

- a) ¿Qué información contiene la Tabla de Símbolos (*Symbol Table*) y la Tabla de Reubicación (*Relocation Table*) emitidas por el ensamblador en un archivo objeto relocable ELF?
- b) ¿Por qué una instrucción de salto relativo (*B etiqueta*) requiere reubicación durante el enlazado si el destino se encuentra en otro archivo fuente compilado por separado?

8.2 Fuente 2: Harris & Harris — *Digital Design and Computer Architecture (ARM Edition)*

Problema H&H 6.1: Principios de Diseño en la Arquitectura ARM (Capítulo 6)

Demuestre cómo la arquitectura ARM aplica los cuatro principios clásicos de diseño de computadores:

- a) *La regularidad favorece la simplicidad (Regularity supports simplicity).*
- b) *Hacer el caso común rápido (Make the common case fast).*
- c) *Menor es más rápido (Smaller is faster).*
- d) *Un buen diseño exige buenos compromisos (Good design demands good compromises).*

Problema H&H 6.3 & 6.4: Direccionamiento por Bytes vs Palabras de 32 bits (Capítulo 6)

En una memoria direccionable por bytes (*byte-addressable*) con arquitectura de 32 bits:

- a) ¿Cuál es la dirección en bytes de la palabra de memoria número 42 (*Word 42*)?
- b) ¿Cuáles son las direcciones en bytes de los 4 bytes individuales que integran la palabra 42?
- c) ¿Por qué una lectura de palabra de 32 bits no alineada (ej. en la dirección `0x0000_002B`) degrada el rendimiento o genera un fallo de alineación en arquitecturas RISC estrictas?

Problema H&H 6.5 & 6.8: Detección de Endianness y Almacenamiento Little-Endian (Capítulo 6)

Considere la variable de 32 bits `uint32_t dato = 0x12345678`; almacenada en la dirección de memoria `0x2000_0000`.

- Muestre el contenido byte a byte en las direcciones `0x2000_0000`, `0x2000_0001`, `0x2000_0002` y `0x2000_0003` en una arquitectura Little-Endian (ARM Cortex-M3) vs Big-Endian.
- Analice el siguiente código en C y explique por qué permite detectar en tiempo de ejecución si el procesador es Little-Endian o Big-Endian:

```
uint32_t test = 1;
char *p = (char *)&test;
if (*p == 1) { /* Little-Endian */ } else { /* Big-Endian */ }
```

Problema H&H 6.27: Estándar de Llamadas AAPCS y Preservación de Registros (Capítulo 6)

En el estándar ARM Architecture Procedure Call Standard (AAPCS):

- ¿Qué registros deben ser preservados obligatoriamente por la función invocada (Callee-Saved) antes de modificar sus valores?
- ¿Qué registros son considerados efímeros (Caller-Saved) y pueden ser sobrescritos libremente por una función?
- ¿Por qué este estándar determinó la selección de los 8 registros apilados automáticamente por hardware en el Cortex-M3 al ingresar a una interrupción?

Problema H&H 9.1 & 9.2: Memory-Mapped I/O y Decodificación de Direcciones (Capítulo 9)

- Diseñe conceptualmente el decodificador de direcciones necesario para mapear un periférico de salida de 32 bits (registro de control de LEDs) en el rango de direcciones `0x4001_0800` a `0x4001_080F`.
- Explique la diferencia entre Polling (espera activa), Interrupciones y DMA (Direct Memory Access) para transferir un bloque de 1 KB desde un receptor UART hacia la memoria RAM, comparando el impacto en la ocupación de la CPU.

8.3 Fuente 3: Joseph Yiu — The Definitive Guide to ARM Cortex-M3 and Cortex-M4**Problema Yiu 7.1: Configuración de Prioridades de Grupo y Desalojo en el NVIC (Capítulo 7)**

En un microcontrolador STM32F103 con 4 bits de prioridad implementados:

- Se configura `PRIGROUP = 0b100` (2 bits para Preemption Priority y 2 bits para Sub-priority).
 - Interrupción A (Timer): Preemption Priority = 1, Sub-priority = 2.
 - Interrupción B (USART): Preemption Priority = 2, Sub-priority = 0.
 - Interrupción C (EXTI): Preemption Priority = 1, Sub-priority = 0.
- Si la Interrupción B se está ejecutando en la CPU y ocurre el evento de la Interrupción A, ¿la Interrupción A desaloja a B? Justifique.
 - Si la Interrupción A se está ejecutando y ocurre la Interrupción C, ¿la Interrupción C desaloja a A? Justifique.
 - Si la CPU está en Thread Mode y ocurren simultáneamente A y C, ¿cuál se atiende primero y por qué?

Problema Yiu 7.3: Cálculo de Ahorro de Ciclos con Tail-Chaining (Capítulo 7)

Considere que la entrada a una interrupción (*Stacking + Vector Fetch*) toma 12 ciclos de reloj y la salida (*Unstacking*) toma 12 ciclos de reloj.

- ¿Cuántos ciclos de reloj toma atender secuencialmente dos interrupciones pendientes ISR_1 e ISR_2 en una arquitectura tradicional sin Tail-Chaining?
- ¿Cuántos ciclos toma en ARM Cortex-M3 utilizando la optimización de Tail-Chaining (6 ciclos de transición entre handlers)? ¿Cuál es el porcentaje de ahorro en sobrecarga de CPU?

8.4 Fuente 4: Richard Barry — Mastering the FreeRTOS Real Time Kernel**Problema FreeRTOS 3.1: Planificación Preemptiva e Inanición (Capítulo 3)**

Considere un sistema con tres tareas:

- Tarea 1: Prioridad 3 (Alta), ejecuta un cálculo que dura 5 ms pero contiene un bucle infinito sin llamadas a `vTaskDelay()` ni bloqueo en colas.
 - Tarea 2: Prioridad 2 (Media), ejecuta una máquina de estados periódica.
 - Tarea 3: Prioridad 1 (Baja), procesa eventos de teclado.
- Describa el comportamiento del Scheduler. ¿Llegan a ejecutarse las Tareas 2 y 3? ¿Cómo se denomina este fenómeno?
 - Modifique la estructura de la Tarea 1 para garantizar que el sistema sea determinista y permita la ejecución de las tareas de menor prioridad.

Problema FreeRTOS 4.2: Dimensionamiento de Colas y Copia por Valor (Capítulo 4)

- a) Si se define una estructura *Telemetria_t* (con campos *uint32_t sensorID*, *float valor* y *uint32_t timestamp*) y se crea una cola de 10 elementos mediante `xQueueCreate(10, sizeof(Telemetria_t))`, ¿cuántos bytes de memoria RAM reserva FreeRTOS en el Heap para el buffer de datos de la cola?
- b) Explique por qué si una tarea emisora envía una variable local asignada en su stack (*Telemetria_t datosLocales*; `xQueueSend(q, &datosLocales, 0);`), los datos recibidos por otra tarea son 100 % válidos incluso si la tarea emisora sale inmediatamente de su bloque de código.

9 Módulo 9: Solucionario Oficial de las Actividades de la Bibliografía

En esta sección se presentan las respuestas formales, deducciones matemáticas paso a paso y justificaciones técnicas extraídas directamente de las fuentes bibliográficas oficiales y manuales de soluciones para las actividades del Módulo 8.

9.1 Soluciones: Patterson & Hennessy (Computer Organization and Design)**Solución P&H 1.12: Falacias de Rendimiento**

- a) **Falso.** La frecuencia de reloj por sí sola no determina el rendimiento. Aplicando la ecuación fundamental:

$$\text{Tiempo CPU}_{P_1} = \frac{IC_1 \times CPI_1}{f_1} = \frac{5,0 \times 10^9 \times 0,9}{4 \times 10^9 \text{ Hz}} = \frac{4,5 \times 10^9}{4 \times 10^9} = \mathbf{1,125 \text{ segundos}}$$

$$\text{Tiempo CPU}_{P_2} = \frac{IC_2 \times CPI_2}{f_2} = \frac{1,0 \times 10^9 \times 0,75}{3 \times 10^9 \text{ Hz}} = \frac{0,75 \times 10^9}{3 \times 10^9} = \mathbf{0,250 \text{ segundos}}$$

Conclusión: P_2 es $\frac{1,125}{0,25} = 4,5 \times$ más rápido que P_1 , a pesar de tener una frecuencia de reloj menor (3 GHz vs 4 GHz), debido a que ejecuta muchas menos instrucciones.

- b) Con $IC_1 = 1,0 \times 10^9$:

$$\text{Tiempo CPU}_{P_1} = \frac{1,0 \times 10^9 \times 0,9}{4 \times 10^9} = \mathbf{0,225 \text{ segundos}}$$

Ahora P_1 es más rápido que P_2 (0,225 s vs 0,250 s) por un factor de $\frac{0,250}{0,225} = \mathbf{1,11 \times}$.

Solución P&H 1.14: Mezcla de Instrucciones y CPI Ponderado

- a) **Cantidad total de instrucciones (IC):**

$$IC = (50 + 110 + 80 + 16) \times 10^6 = \mathbf{256 \times 10^6 \text{ instrucciones}}$$

Ciclos de reloj totales:

$$\text{Ciclos} = (50 \times 10^6 \times 1) + (110 \times 10^6 \times 1) + (80 \times 10^6 \times 4) + (16 \times 10^6 \times 2)$$

$$\text{Ciclos} = (50 + 110 + 320 + 32) \times 10^6 = \mathbf{512 \times 10^6 \text{ ciclos}}$$

$$\text{CPI Ponderado} = \frac{512 \times 10^6}{256 \times 10^6} = \mathbf{2,0}$$

$$\text{Tiempo de CPU} = \frac{512 \times 10^6 \text{ ciclos}}{2 \times 10^9 \text{ Hz}} = \mathbf{0,256 \text{ segundos (256 ms)}}$$

- b) Para duplicar la velocidad, el tiempo debe reducirse a 0,128 s, lo que exige un total de 256×10^6 ciclos (reducción de 256 millones de ciclos). Como las instrucciones FP consumen en total solo 50×10^6 ciclos, incluso si su CPI fuera 0 (tiempo cero), solo se ahorrarían 50 millones de ciclos. **Es físicamente imposible alcanzar la meta optimizando únicamente las instrucciones FP** (demostración de la Ley de Amdahl).
- c) Las instrucciones L/S consumen actualmente 320×10^6 ciclos. Para ahorrar 256 millones de ciclos, deben pasar a consumir $320 - 256 = 64 \times 10^6$ ciclos.

$$CPI_{L/S \text{ nuevo}} = \frac{64 \times 10^6 \text{ ciclos}}{80 \times 10^6 \text{ instrucciones}} = \mathbf{0,8}$$

El CPI de L/S debe reducirse de 4 a 0,8 (una mejora de $\frac{4}{0,8} = 5 \times$).

- d) Nuevos ciclos:

$$\text{Ciclos}_{\text{nuevo}} = (50 \times 0,6) + (110 \times 0,6) + (80 \times 2,8) + (16 \times 1,4) = 30 + 66 + 224 + 22,4 = \mathbf{342,4 \times 10^6}$$

$$\text{Tiempo}_{\text{nuevo}} = \frac{342,4 \times 10^6}{2 \times 10^9} = \mathbf{0,1712 \text{ segundos}}$$

Mejora global: $\frac{0,256}{0,1712} = \mathbf{1,495 \times}$ (el programa corre un 49,5 % más rápido).

Solución P&H 1.15: Ley de Amdahl con Sobrecarga

a) La fórmula del tiempo con p procesadores es $T(p) = \frac{100}{p} + 4$ segundos:

- $p = 2 \rightarrow T(2) = \frac{100}{2} + 4 = 54 \text{ s} \rightarrow S = \frac{100}{54} = 1,85 \times$
- $p = 4 \rightarrow T(4) = \frac{100}{4} + 4 = 29 \text{ s} \rightarrow S = \frac{100}{29} = 3,45 \times$
- $p = 8 \rightarrow T(8) = \frac{100}{8} + 4 = 16,5 \text{ s} \rightarrow S = \frac{100}{16,5} = 6,06 \times$
- $p = 16 \rightarrow T(16) = \frac{100}{16} + 4 = 10,25 \text{ s} \rightarrow S = \frac{100}{10,25} = 9,76 \times$
- $p = 64 \rightarrow T(64) = \frac{100}{64} + 4 = 1,56 + 4 = 5,56 \text{ s} \rightarrow S = \frac{100}{5,56} = 17,98 \times$

b) Límite asintótico con infinitos procesadores:

$$\lim_{p \rightarrow \infty} T(p) = \lim_{p \rightarrow \infty} \left(\frac{100}{p} + 4 \right) = 0 + 4 = 4 \text{ segundos}$$

$$\text{Aceleración Máxima Teórica } S_{\max} = \frac{100 \text{ s}}{4 \text{ s}} = 25 \times$$

Solución P&H 2.12: Tabla de Símbolos y Reubicación

- a) ■ **Tabla de Símbolos (Symbol Table):** Lista todos los identificadores con visibilidad global (nombres de funciones exportadas, variables globales) junto con su sección y dirección relativa (offset) dentro del archivo objeto.
- **Tabla de Reubicación (Relocation Table):** Lista las líneas de código que contienen direcciones absolutas o saltos a símbolos que no pudieron ser resueltos durante el ensamblado porque pertenecen a otros archivos objeto.
- b) El ensamblador genera la instrucción de salto B dejando un campo de offset en cero. El *Linker*, tras agrupar todas las secciones .text y asignar las direcciones finales de memoria, calcula la distancia exacta en bytes (Destino – Origen – 4) y parchea el opcode binario con el desplazamiento relativo final.

9.2 Soluciones: Harris & Harris (Digital Design and Computer Architecture)**Solución H&H 6.1: Principios de Diseño en ARM**

- a) **Regularidad favorece simplicidad:** Todas las instrucciones de procesamiento de datos tienen 3 operandos (dos fuentes y un destino: ADD Rd, Rn, Rm).
- b) **Hacer el caso común rápido:** La arquitectura incluye un *Barrel Shifter* por hardware que permite desplazar un operando en el mismo ciclo de reloj de una suma o resta.
- c) **Menor es más rápido:** Un banco de solo 16 registros visibles (R0-R15) permite decodificaciones ultrarrápidas y acceso en fracciones de nanosegundo.
- d) **Buen diseño exige buenos compromisos:** El conjunto de instrucciones **Thumb-2** compromete parte de la flexibilidad de 32 bits a cambio de instrucciones de 16 bits que reducen el tamaño de código en un 35 % con solo un 2 % de pérdida de rendimiento.

Solución H&H 6.3 & 6.4: Direccionamiento por Bytes

a) Cada palabra de 32 bits contiene 4 bytes. La dirección del primer byte de la palabra 42 es:

$$\text{Dirección en Bytes} = 42 \times 4 = 168 = 0x0000_00A8$$

- b) Los 4 bytes ocupan las direcciones consecutivas: 0x0000_00A8, 0x0000_00A9, 0x0000_00AA y 0x0000_00AB.
- c) La memoria física está organizada en bancos de 32 bits con líneas de dirección alineadas a múltiplos de 4 (bits [1:0] = 00). Acceder a una dirección no alineada como 0x2B exige que el hardware realice dos accesos sucesivos a memoria y recombine los bytes mediante desplazamientos, duplicando el tiempo de acceso o disparando un UsageFault si la bandera UNALIGN_TRP está activa.

Solución H&H 6.5 & 6.8: Endianness en Memoria

a) Para el valor 0x12345678 (MSB = 0x12, LSB = 0x78):

Dirección	Little-Endian (ARM)	Big-Endian
0x2000_0000 (Base)	0x78 (Byte menos significativo)	0x12 (Byte más significativo)
0x2000_0001	0x56	0x34
0x2000_0002	0x34	0x56
0x2000_0003	0x12 (Byte más significativo)	0x78 (Byte menos significativo)

- b) Al hacer un cast del puntero de 32 bits a un puntero de caracteres (char *p), *p desreferencia únicamente el primer byte en la dirección base. En Little-Endian el valor 1 reside en el primer byte (*p == 1); en Big-Endian el primer byte contiene 0.

Solución H&H 6.27: Estándar AAPCS y Stacking

- a) **Callee-Saved (Preservados por el receptor):** R4 – R11, SP (R13). Si una función necesita usarlos, debe guardarlos en el stack con PUSH y restaurarlos con POP antes de retornar.
- b) **Caller-Saved (Efímeros / Parámetros):** R0 – R3 (argumentos y retorno), R12 (scratch register), LR (R14), PC (R15) y xPSR.
- c) Debido a que cualquier función escrita en C puede sobrescribir los registros *Caller-Saved* sin restaurarlos, el hardware del Cortex-M3 fue diseñado para apilar automáticamente por hardware exactamente esos 8 registros (R0–R3, R12, LR, PC, xPSR) en 12 ciclos, permitiendo escribir ISRs directamente como funciones ordinarias en C.

9.3 Soluciones: Joseph Yiu (*The Definitive Guide to ARM Cortex-M3/M4*)**Solución Yiu 7.1: Prioridades y Desalojo en el NVIC**

- a) **Sí desaloja.** La Interrupción A tiene Preemption Priority = 1, que es numéricamente menor (mayor prioridad de grupo) que la Interrupción B (Preemption Priority = 2).
- b) **NO desaloja.** La Interrupción C y la Interrupción A tienen la misma Preemption Priority (1). Aunque C tiene una menor Sub-priority (0 vs 2), la subprioridad NO otorga derecho de desalojo sobre una ISR que ya está corriendo. C quedará en estado Pending hasta que A termine.
- c) Se atiende primero la **Interrupción C**. Al ocurrir simultáneamente en Thread Mode, tienen igual Preemption Priority (1), por lo que el desempate lo define la Sub-priority (C tiene subprioridad 0, más prioritaria que el 2 de A).

Solución Yiu 7.3: Ahorro de Ciclos con Tail-Chaining

- a) **Sin Tail-Chaining:**

$$\text{Ciclos} = \text{Stacking}_1(12) + \text{ISR}_1 + \text{Unstacking}_1(12) + \text{Stacking}_2(12) + \text{ISR}_2 + \text{Unstacking}_2(12)$$

Sobrecarga total de hardware = 12 + 12 + 12 + 12 = **48** ciclos.

- b) **Con Tail-Chaining en Cortex-M3:**

$$\text{Ciclos} = \text{Stacking}_1(12) + \text{ISR}_1 + \text{Tail-Chain}(6) + \text{ISR}_2 + \text{Unstacking}_2(12)$$

Sobrecarga total de hardware = 12 + 6 + 12 = **30** ciclos.

Ahorro Absoluto = 48 – 30 = **18** ciclos (37,5 % de reducción en la sobrecarga de interrupción).

9.4 Soluciones: Richard Barry (*Mastering the FreeRTOS Real Time Kernel*)**Solución FreeRTOS 3.1: Inanición de Tareas**

- a) El Scheduler preemptivo ejecutará continuamente la Tarea 1 por ser la de mayor prioridad (Prioridad 3). Como nunca entra en estado Blocked, **las Tareas 2 y 3 jamás recibirán tiempo de CPU**. Este fenómeno se denomina **Inanición de Tareas (Task Starvation)**.
- b) Debe reestructurarse la Tarea 1 para que, tras completar su cálculo de 5 ms, ceda voluntariamente el control de la CPU bloqueándose mediante una función de retardo:

```
void vTarea1(void *pvParameters) {
    for(;;) {
        EjecutarCalculoPesado(); // 5 ms de trabajo
        vTaskDelay(pdMS_TO_TICKS(95)); // Se bloquea 95 ms permitiendo correr a Tareas 2 y 3
    }
}
```

Solución FreeRTOS 4.2: Dimensionamiento de Colas y Copia por Valor

- a) Tamaño de la estructura: $\text{sizeof}(\text{Telemetria_t}) = 4 + 4 + 4 = 12$ bytes.

Memoria para datos = 10×12 bytes = **120** bytes en el Heap (+estructura de control de la cola ~ 76 bytes).

- b) Porque `xQueueSend()` realiza una copia profunda byte a byte (*memcpy*) desde la dirección de la variable local hacia el buffer interno de la cola en el Heap de FreeRTOS. Cuando la función emisora retorna y destruye su stack frame, los datos ya están resguardados en la memoria persistente del kernel.